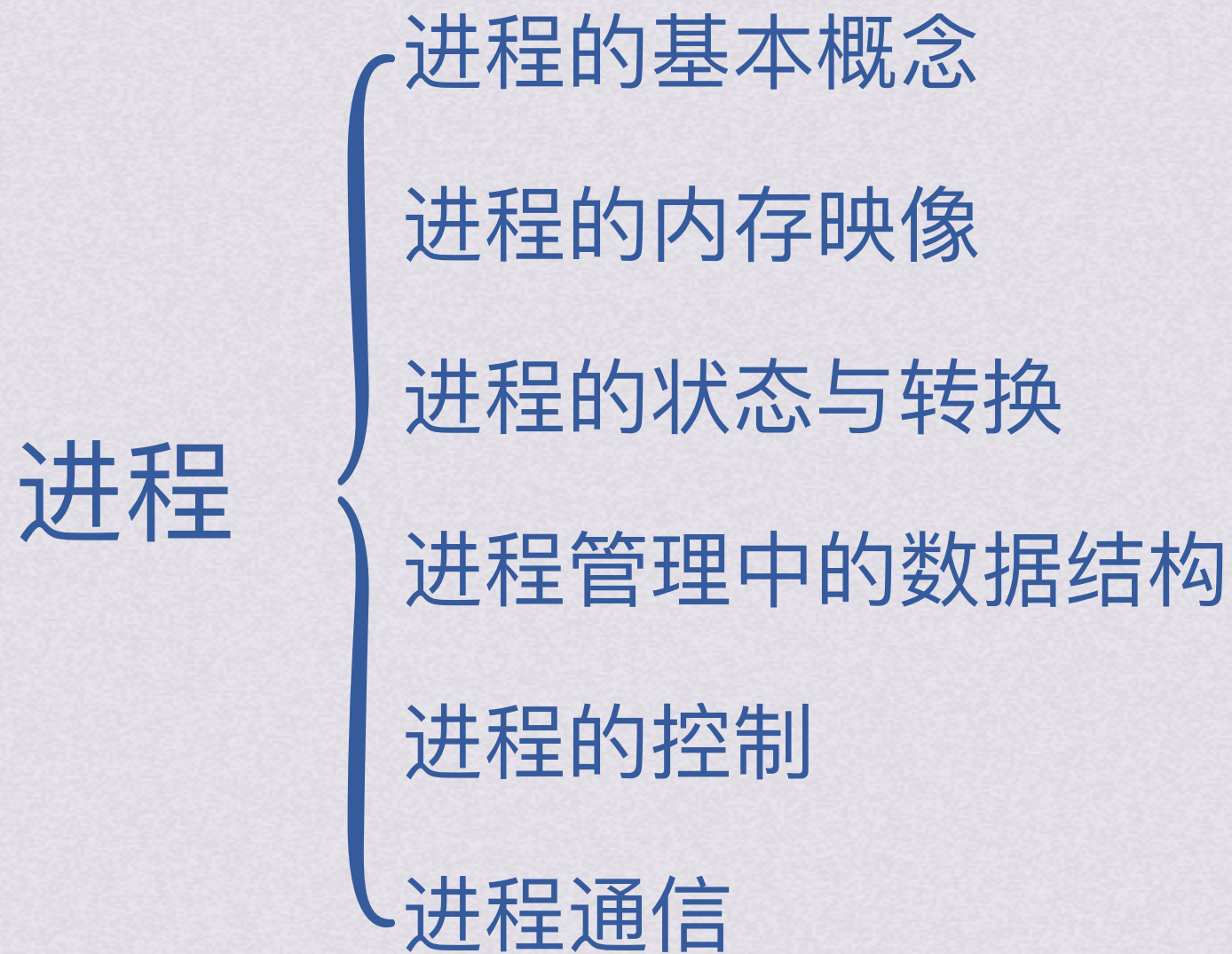


bok25

**基
礎**

課、操作系統





进程的基本概念

进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的基本概念 定义

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令（或一些静态数据）。操作系统让这些字节运行起来，让程序发挥作用

01000101...

程序

本身作为一串二进制指令/数据无意义



进程的基本概念 定义

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令（或一些静态数据）。操作系统让这些字节运行起来，让程序发挥作用

01000101...

程序

本身作为一串二进制指令/数据无意义



操作系统

进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信

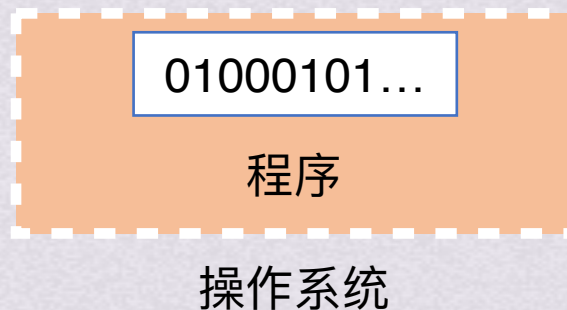


进程的基本概念 定义

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令（或一些静态数据）。操作系统让这些字节运行起来，让程序发挥作用

程序本身作为一串二进制指令/数据无意义

操作系统让程序运行起来后，程序就可以发挥作用



{ 进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的基本概念 特征

1. **动态性**: 进程是程序的执行过程, 动态性是进程最基本的特征。进程由创建而产生, 由调度而执行, 由撤销而消亡。所以进程有一定的生命期, 程序只是静态的有序指令的集合
2. **并发性**: 多个进程共存于内存中, 在一段时间内同时执行。所以并发也是进程的重要特征
3. **独立性**: 进程是一个能独立运行, 独立获得资源, 独立接受调度的基本单位
4. **异步性**: 进程都是按照异步方式运行的, 即按各自独立、不可预知的速度向前推进

进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行

输入进程在输入第一个程序后，在计算程序对该程序进行计算的同时，可由输入进程再输入第二个程序从而使第一个程序的计算操作可与第二个程序的输入操作并发执行

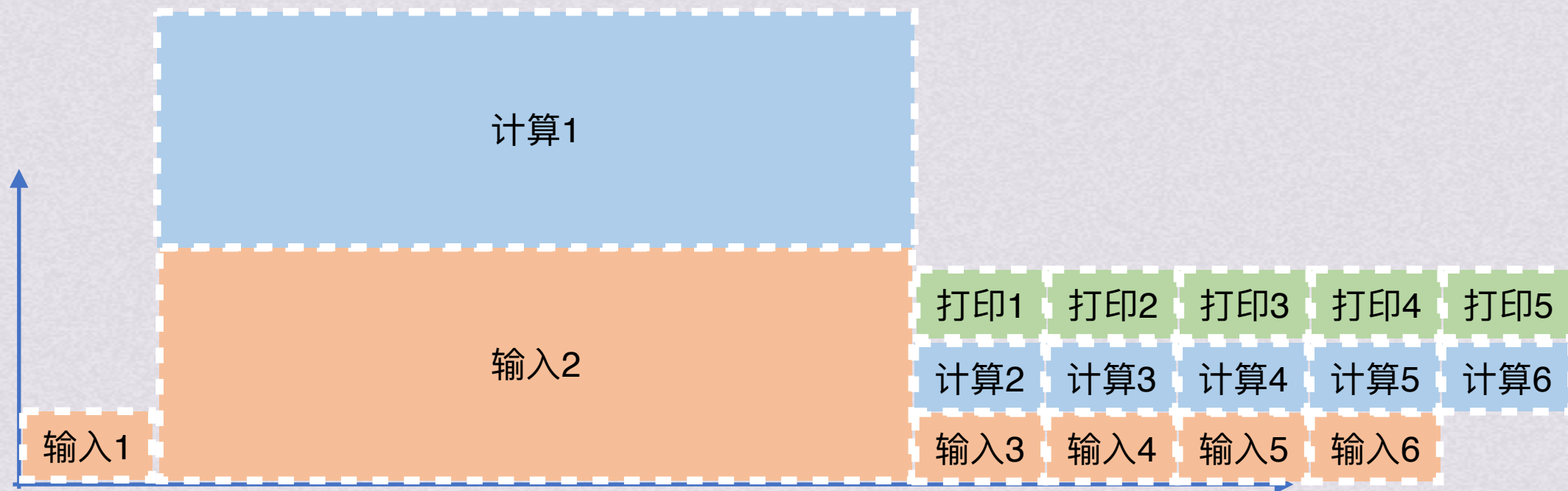




进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行

输入进程在输入第一个程序后，在计算程序对该程序进行计算的同时，可由输入进程再输入第二个程序从而使第一个程序的计算操作可与第二个程序的输入操作并发执行



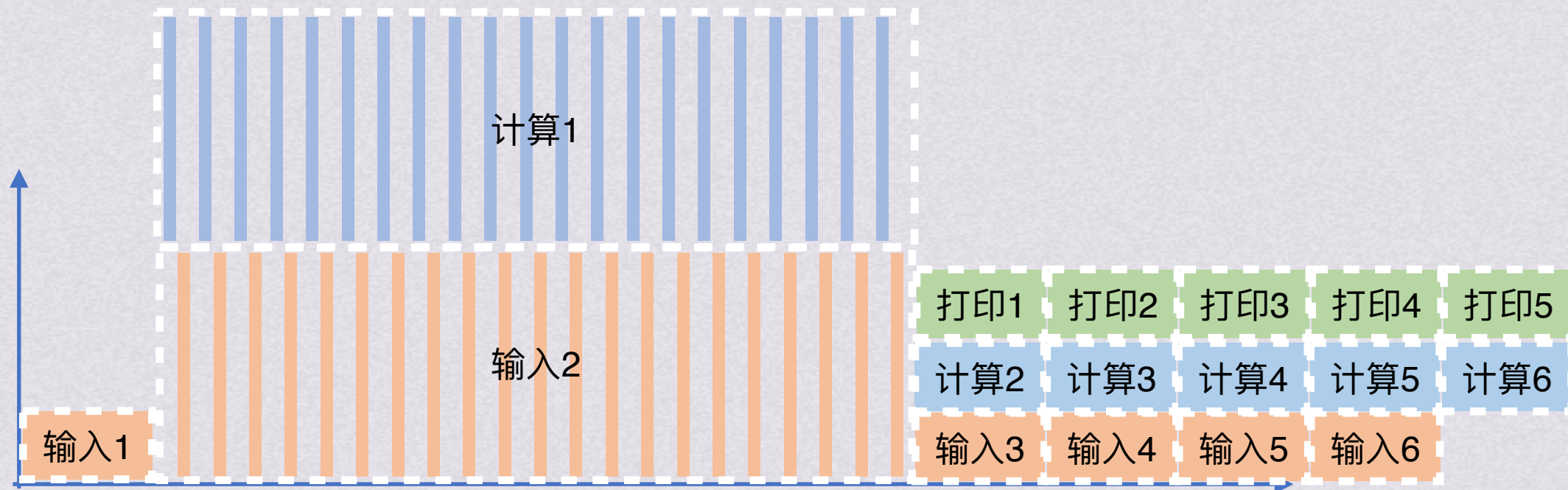
进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行

输入进程在输入第一个程序后，在计算程序对该程序进行计算的同时，可由输入进程再输入第二个程序从而使第一个程序的计算操作可与第二个程序的输入操作并发执行



进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行

输入进程在输入第一个程序后，在计算程序对该程序进行计算的同时，可由输入进程再输入第二个程序从而使第一个程序的计算操作可与第二个程序的输入操作并发执行





进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律



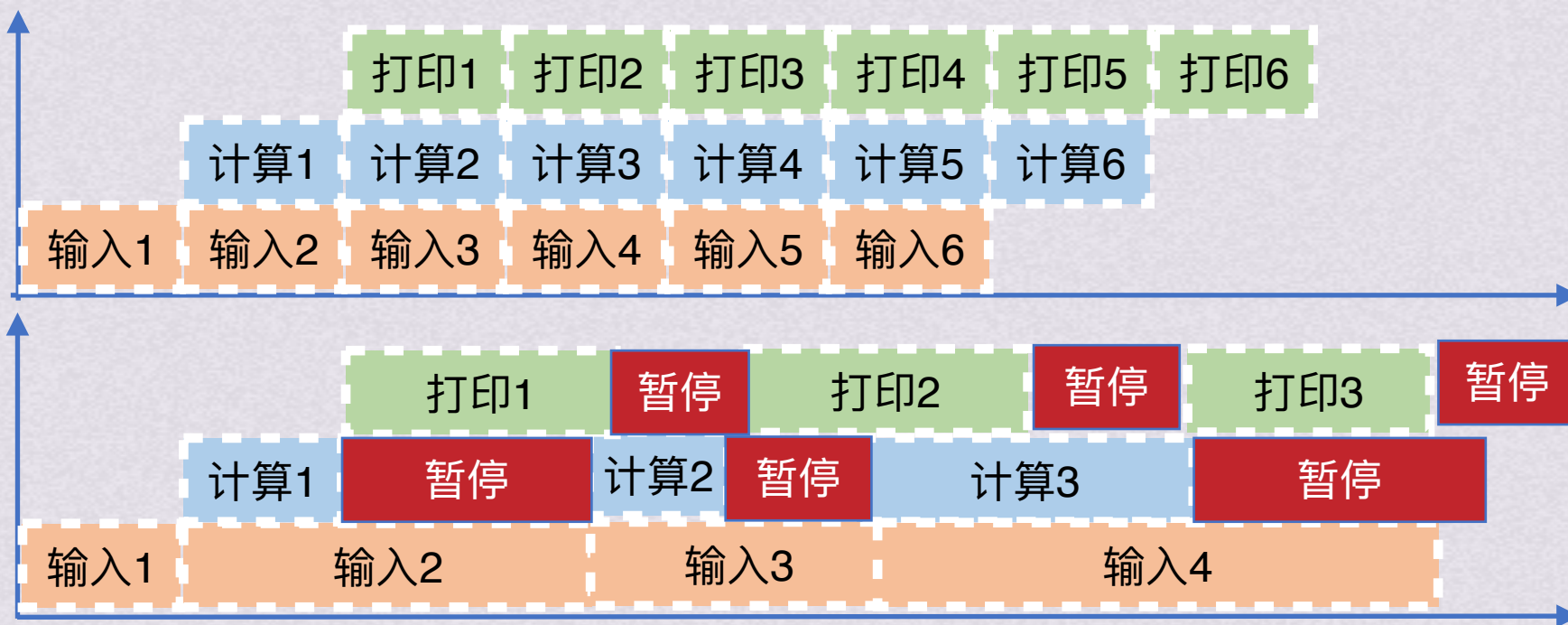


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律



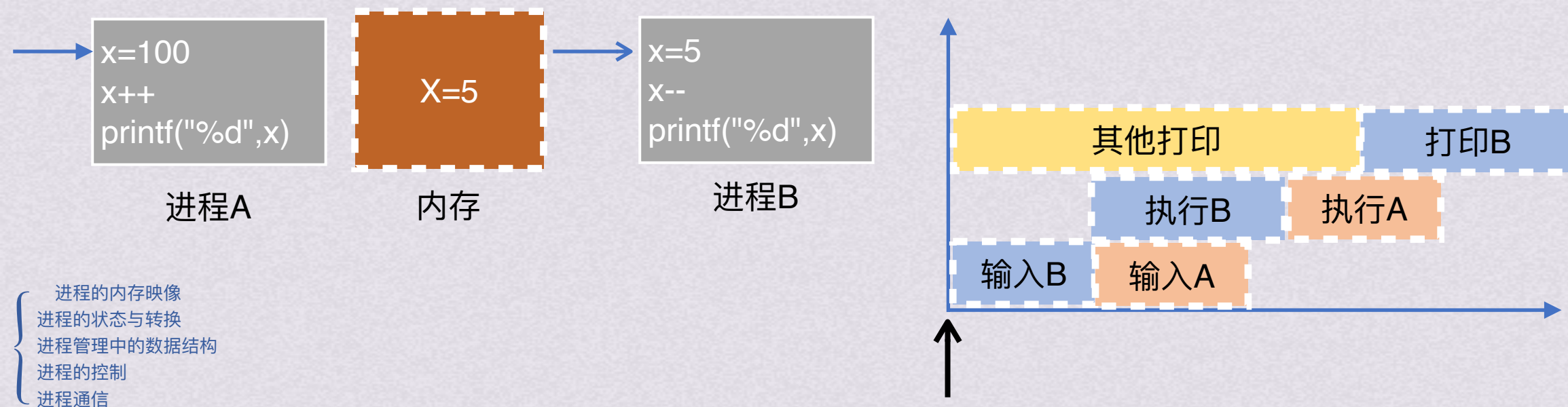


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



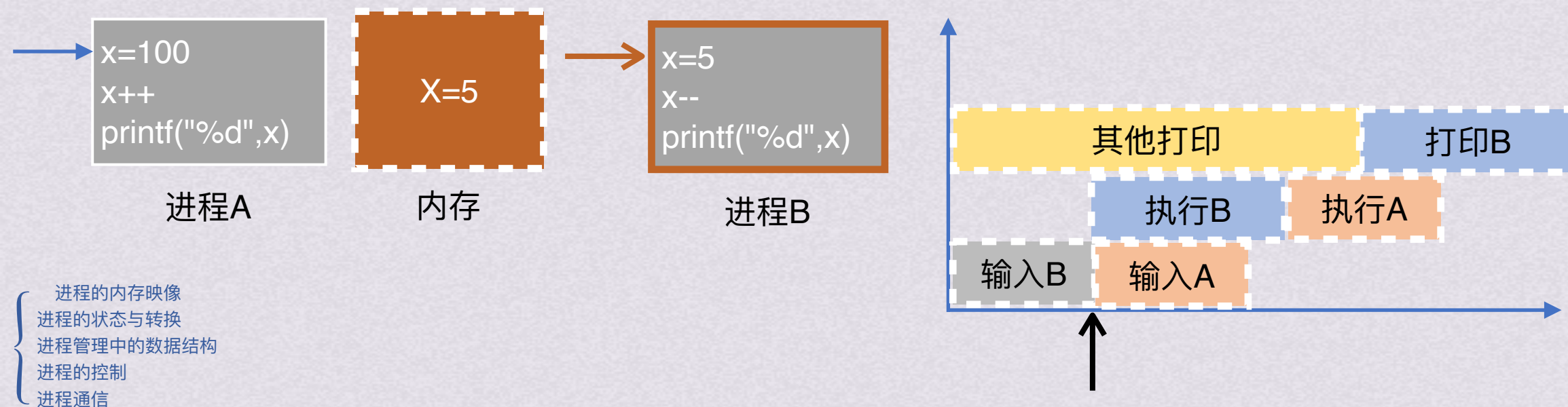


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



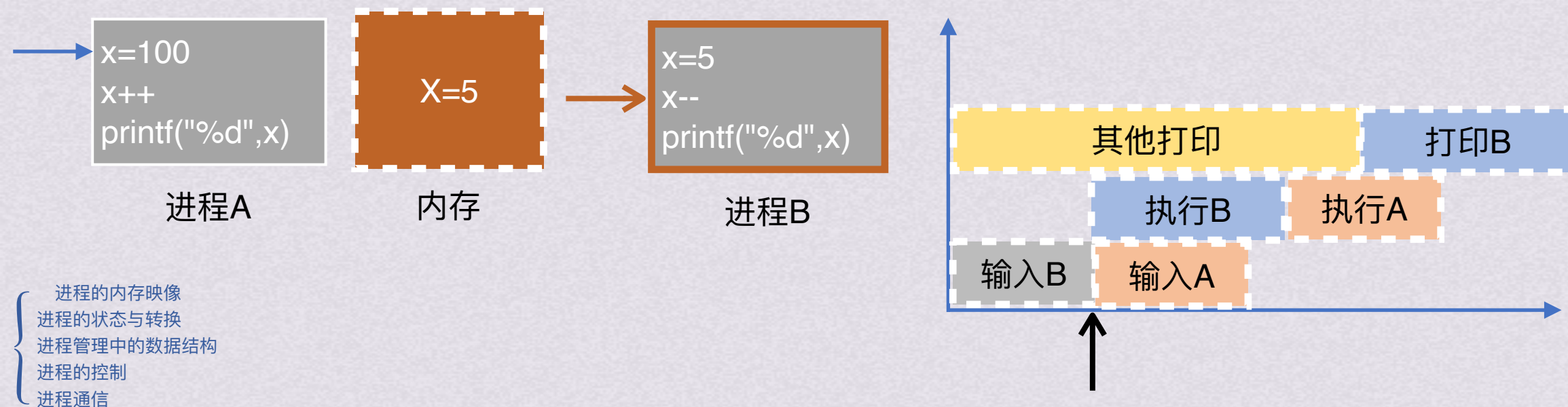


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



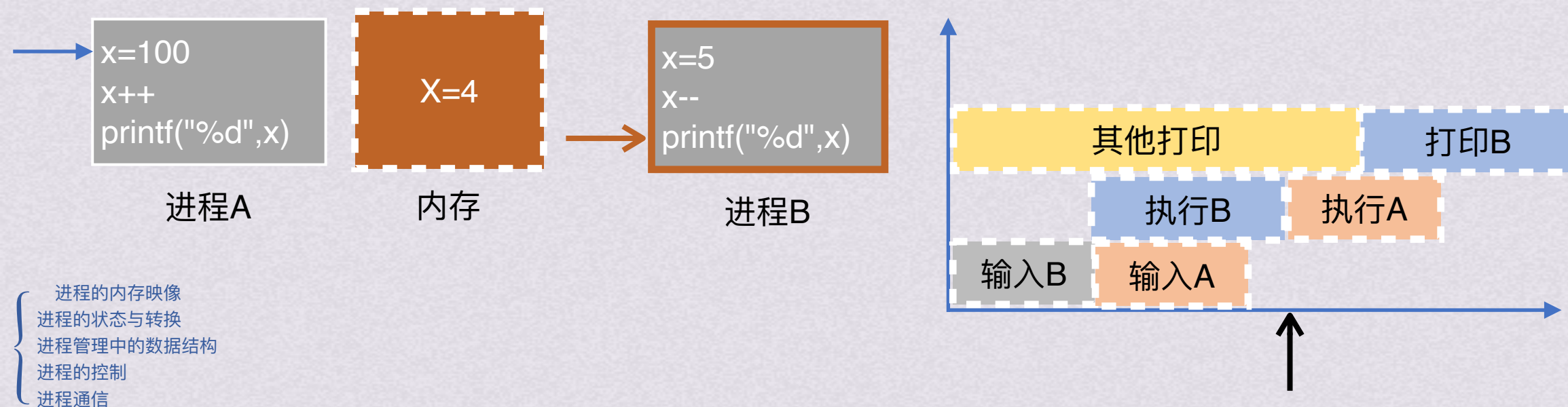


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



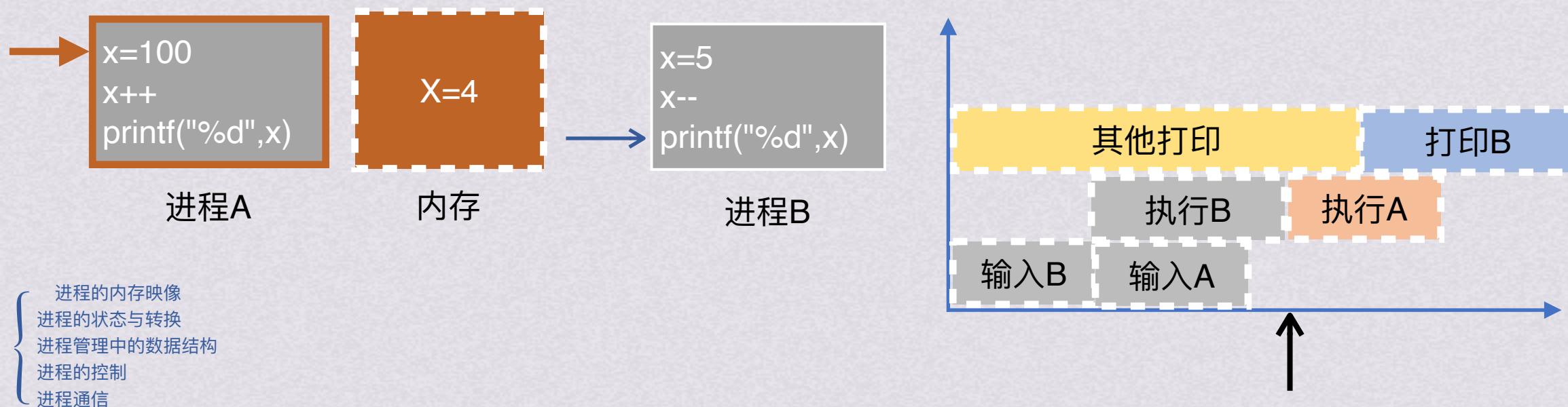


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



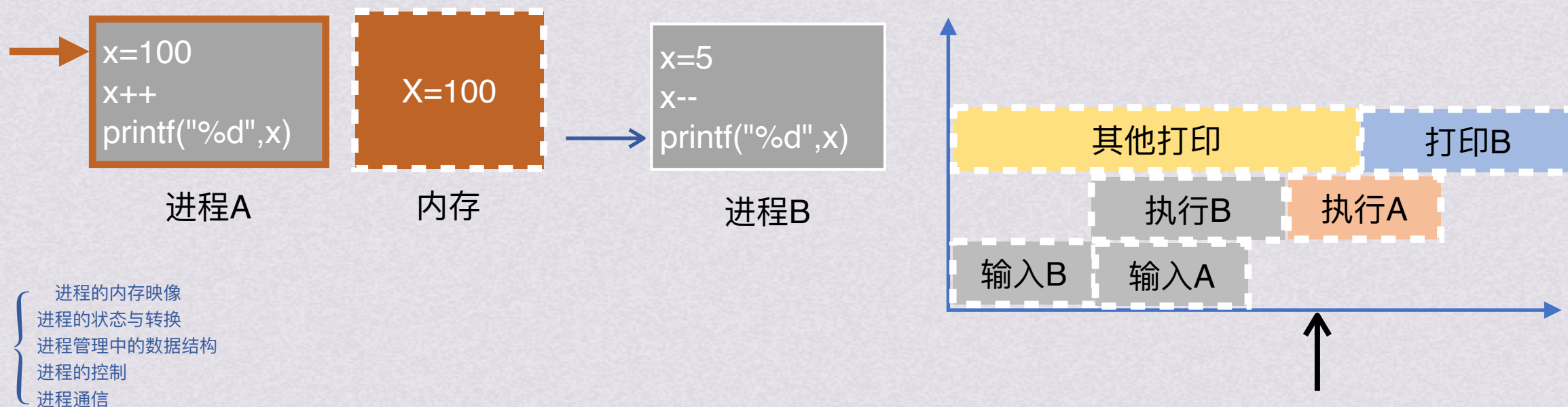


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



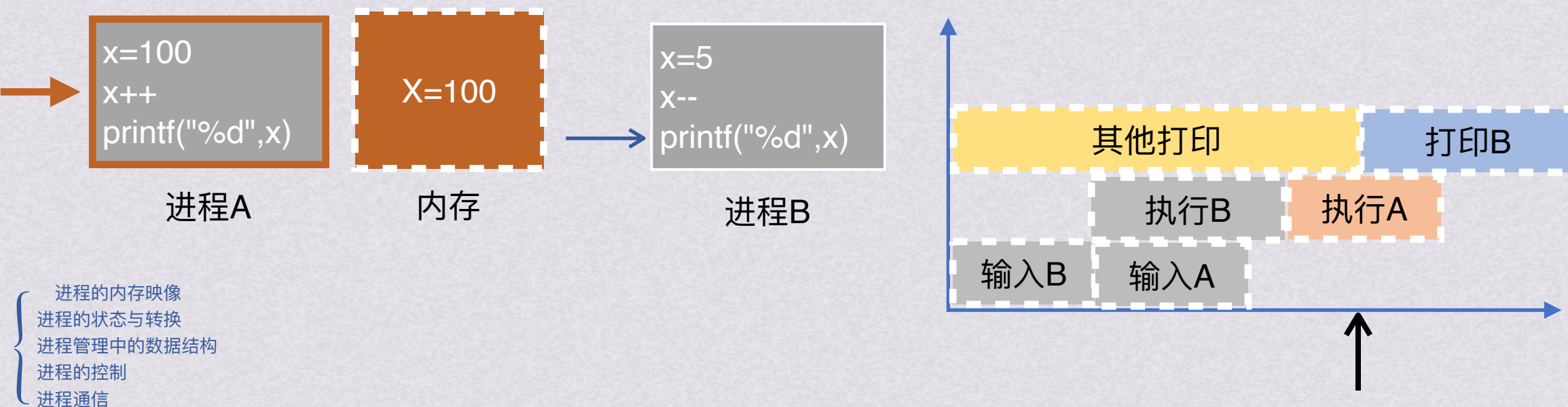


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。



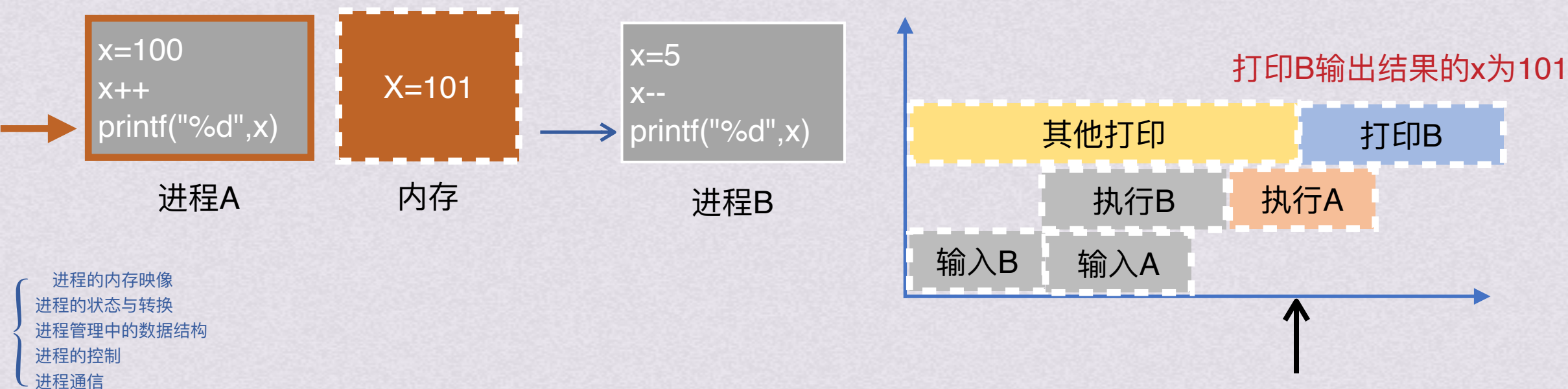


进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发带来了更高的系统吞吐量，也带来了新的问题：

- 1.间断性：本例的输入->计算->打印流程，在并发执行时，若每一环节的时间不相同，那必然会有需要等待的环节，致使在这些并发执行的进程之间，形成了相互制约的关系，导致并发进程具有“执行—暂停—执行”这种间断性的活动规律
- 2.失去封闭性：进程在并发执行时，是多个进程共享系统中的各种资源，因而这些资源的状态将由多个进程来改变，致使程序的运行失去了封闭性。
- 3.不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。





进程的基本概念 特征

2.并发性：多个进程共存于内存中，在一段时间内同时执行。所以并发也是进程的重要特征。未建立程序控制块PCB的程序是不能作为一个独立的单位参与并发执行的

并发进程会失去封闭性，指并发进程共享变量，执行结果与速度有关

程序封闭性：是指进程执行的结果只取决于进程本身，不受外界影响。

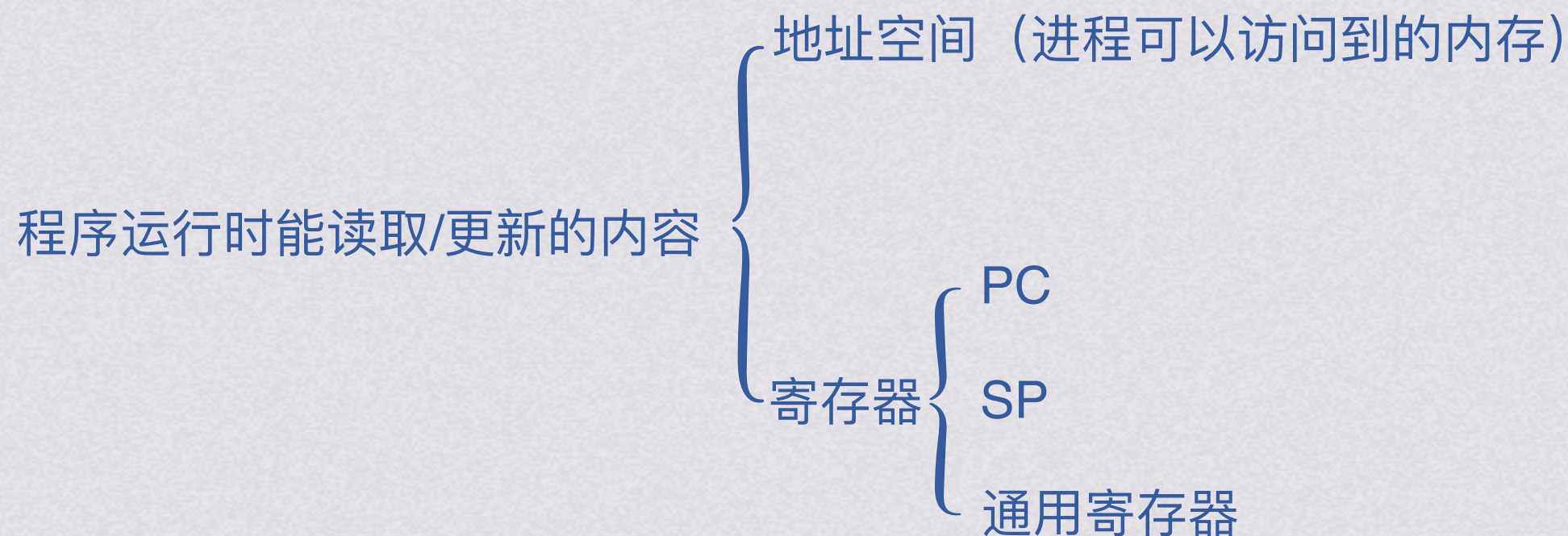
进程在执行过程中不管是不停顿的执行，还是走走停停，进程的执行速度不会改变它的执行结果。

可再现性：是指当程序重复执行多次时，必将得到相同结果。

程序并发运行和进程的并发运行都失去了封闭性和可再现性。



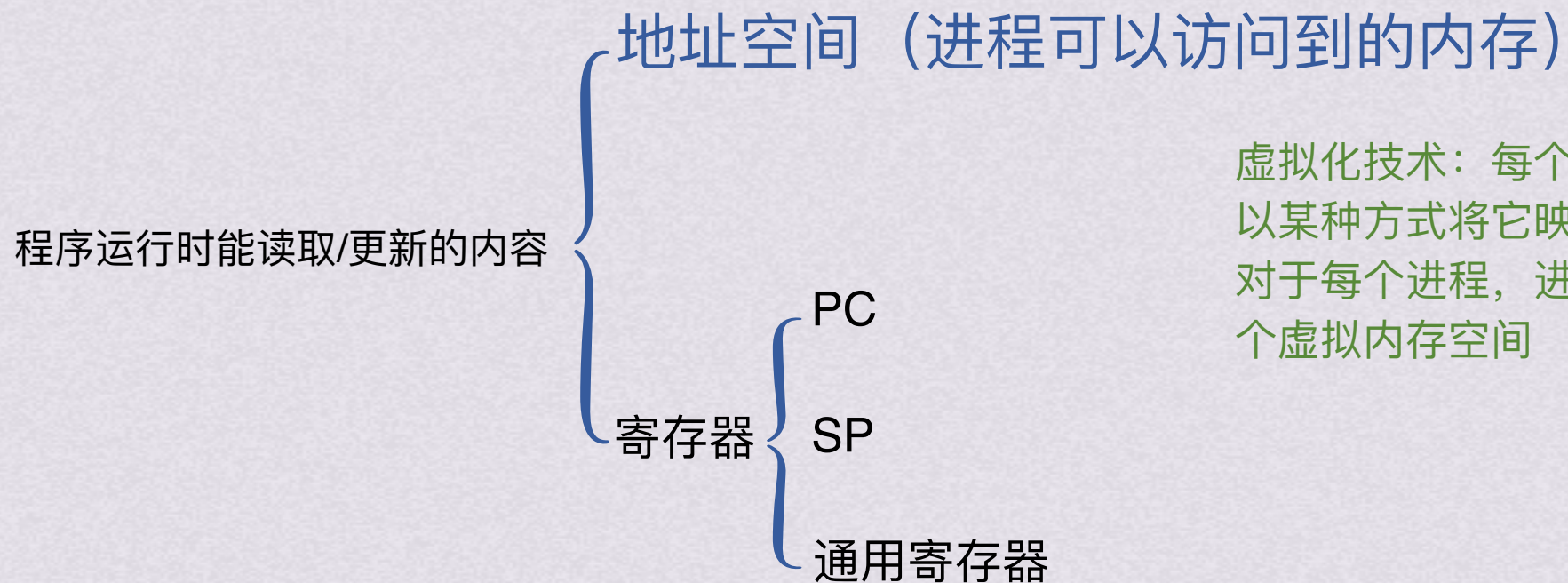
进程的基本概念 机器状态



- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的基本概念 机器状态



虚拟化技术：每个进程访问自己的地址空间，OS会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个CPU和整个虚拟内存空间

- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的基本概念 机器状态

虚拟化技术：每个进程访问自己的地址空间，OS会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个CPU和整个虚拟内存空间

进程的内存映像

进程的状态与转换
进程管理中的数据结构
进程的控制
进程通信



进程的内存映像

虚拟化技术：每个进程访问自己的地址空间，OS会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个CPU和整个虚拟内存空间



进程的内存映像

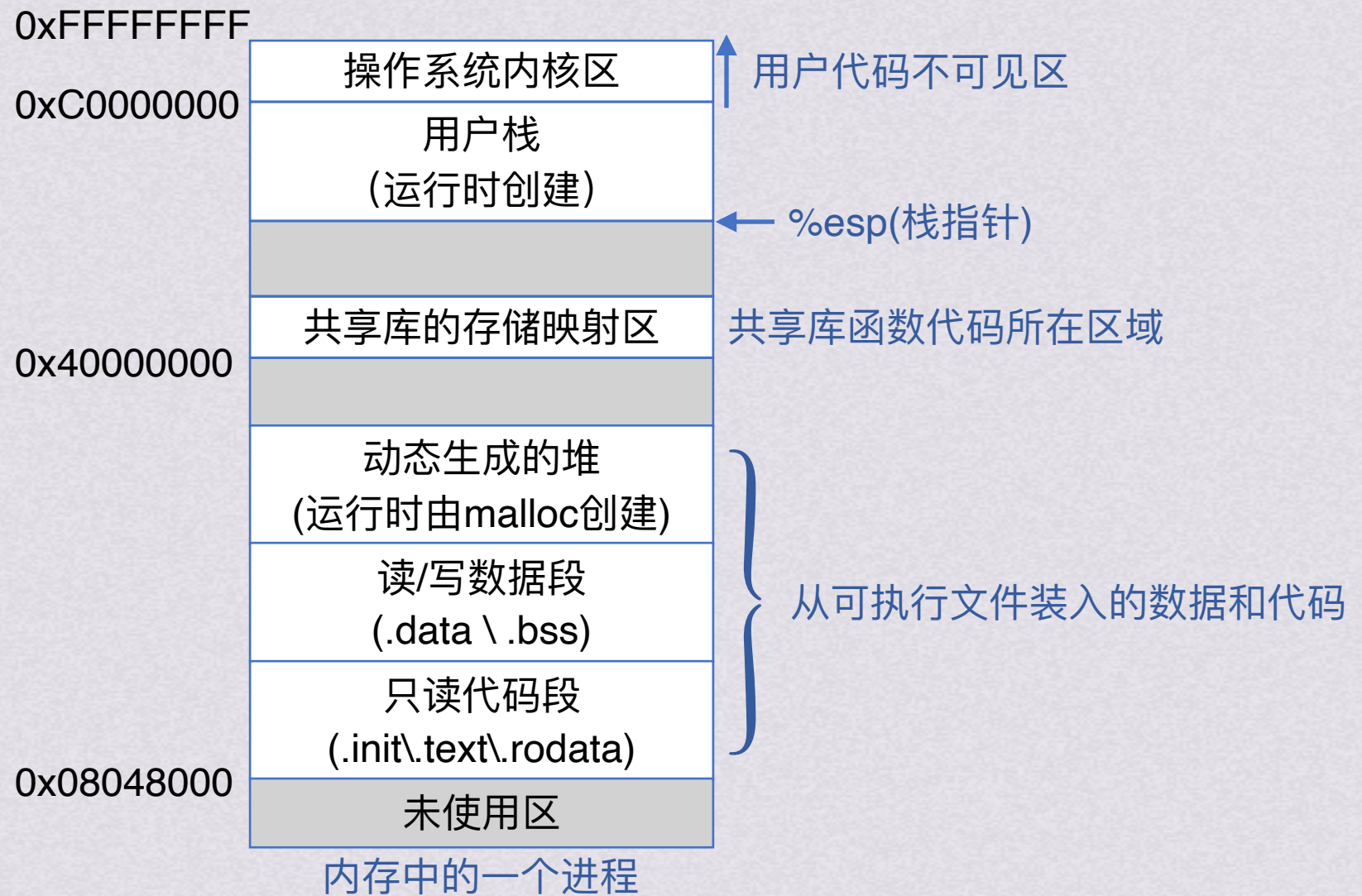
请务必学习完虚拟存储器后再观看本部分

虚拟化技术：每个进程访问自己的地址空间，OS会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个CPU和整个虚拟内存空间

进程的内存映像

请务必学习完虚拟存储器后再观看本部分

程序调入内存运行时，就构成了进程的内存映像



- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信

进程的内存映像

请务必学习完虚拟存储器后再观看本部分

程序调入内存运行时，就构成了进程的内存映像

0xFFFFFFFF

0xC0000000

32位程序中虚拟地址为32个bit，即程序可以访问到的地址空间最大是0xFFFF FFFF

为每个进程创建巨大的、私有的虚拟内存 (virtualmemory) 的假象, 这种地址空间 (addressspace) 的抽象让每个程序好像拥有自己的内存，而实际上操作系统秘密地让多个地址空间复用物理内存(或者磁盘)。

0x08048000



↑ 用户代码不可见区

← %esp(栈指针)

共享库函数代码所在区域

} 从可执行文件装入的数据和代码

内存中的一个进程

- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信

进程的内存映像

请务必学习完虚拟存储器后再观看本部分

程序调入内存运行时，就构成了进程的内存映像
一个进程的内存映像一般有几个要素：

代码段：即程序的二进制代码，代码段是只读的，可被多个进程共享

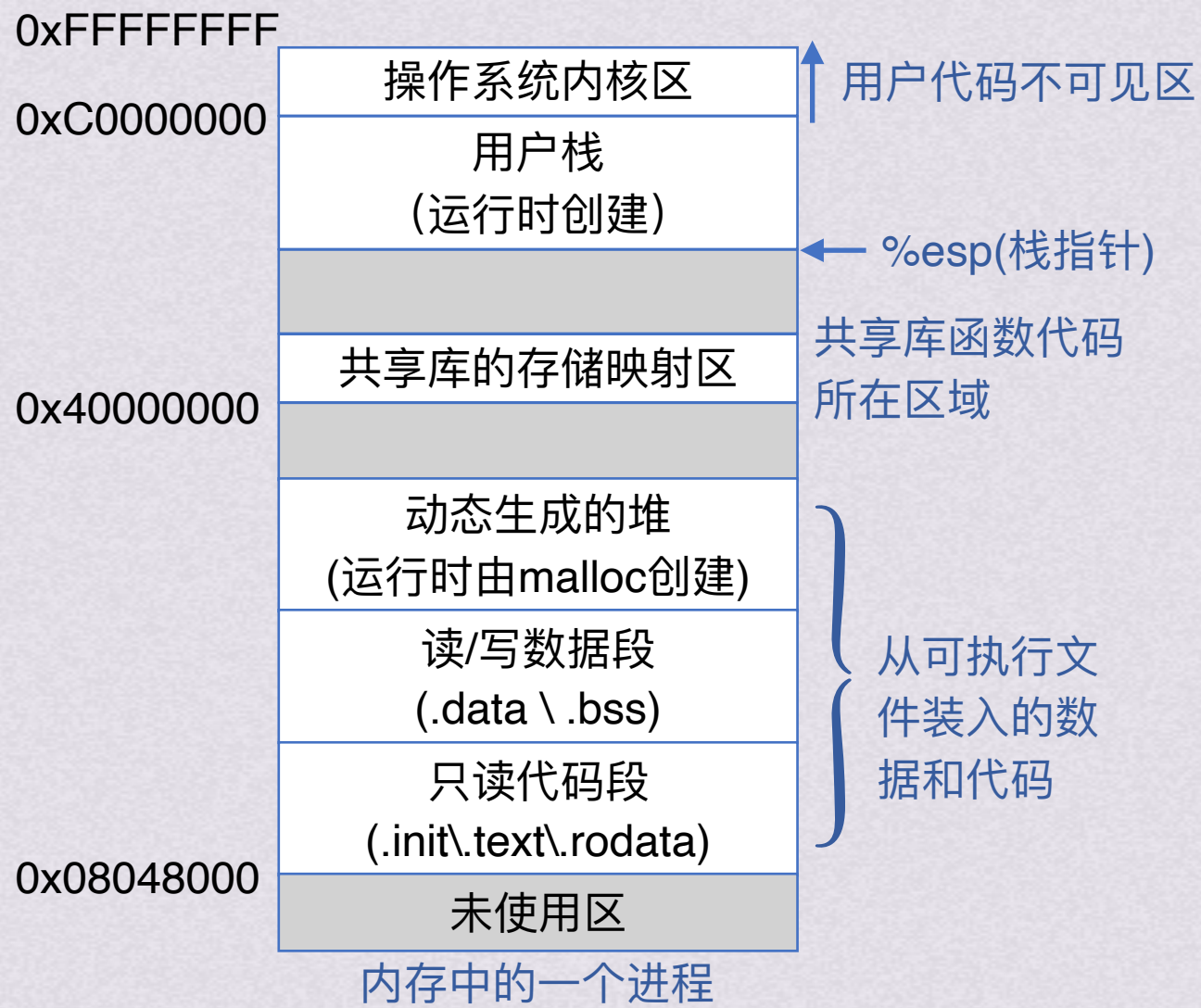
数据段：即程序运行时加工处理的对象，包括全局变量和静态变量

进程控制块PCB：放在系统区

堆：用来存放动态分配的变量，通过调用malloc函数动态的向高地址分配空间

栈：用来实现函数调用，从用户空间的最大地址向低地址方向增长

- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



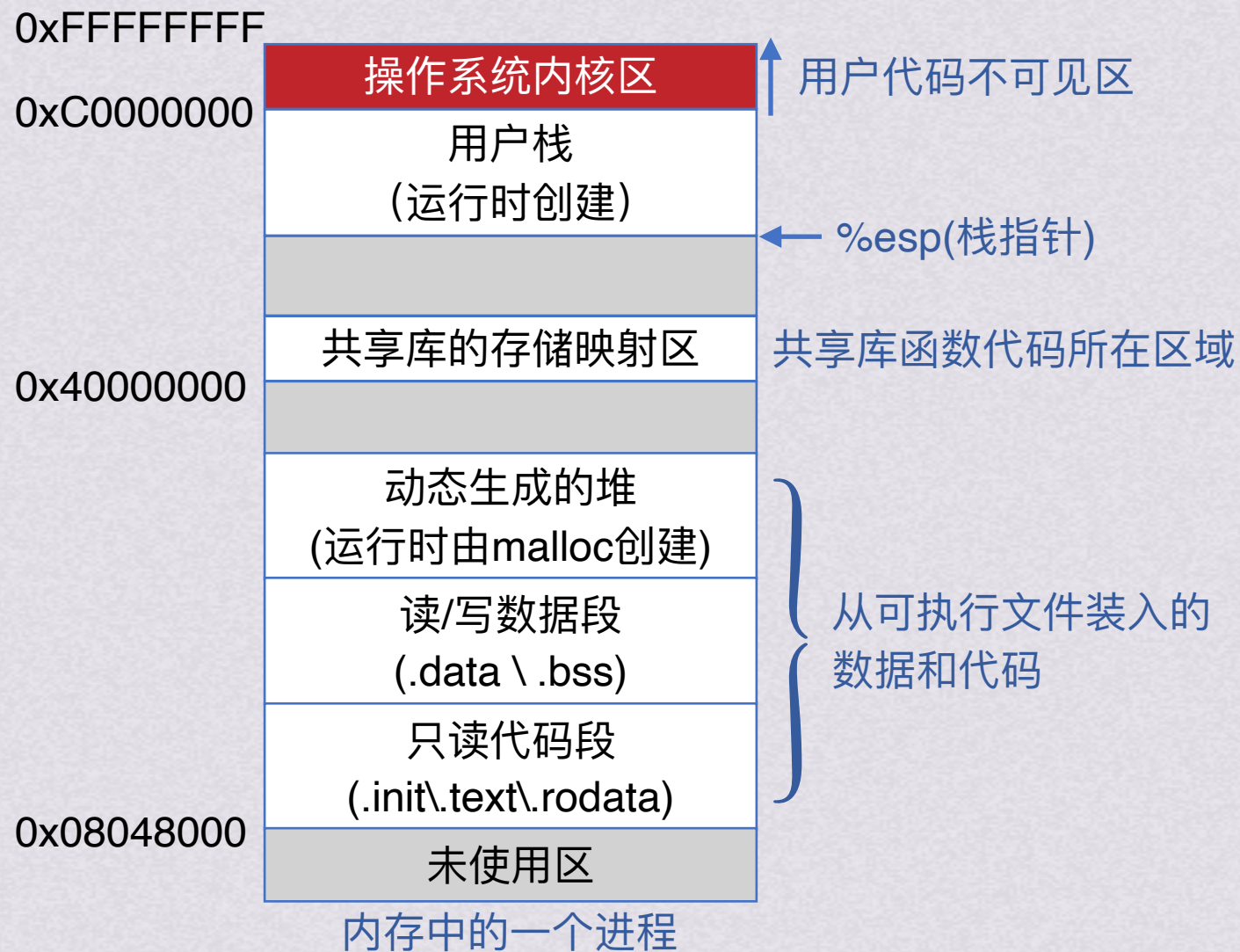


进程的内存映像

请务必学习完虚拟存储器后再观看本部分

操作系统内核区：将操作系统内核映射到这个区域，这个区域里的内容对进程不可见。对于虚拟存储系统中每个进程，它都以为整个系统中只有它自己和OS内核

并且0xC000 0000以上是OS内核的活动空间



进程的基本概念

进程的状态与转换

进程管理中的数据结构

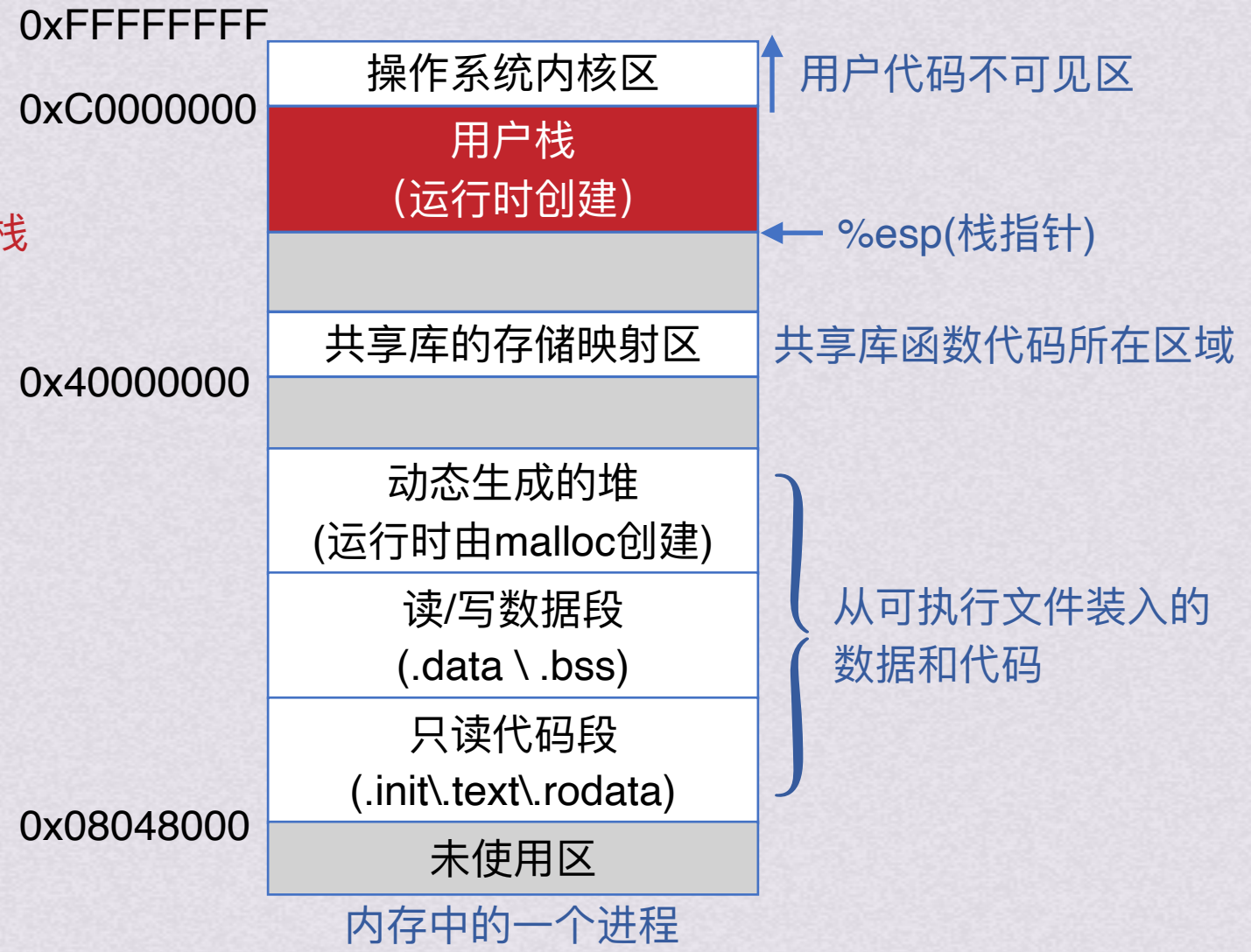
进程的控制

进程通信

进程的内存映像

请务必学习完虚拟存储器后再观看本部分

用户栈：存局部变量，函数调用传参过程也会用到栈



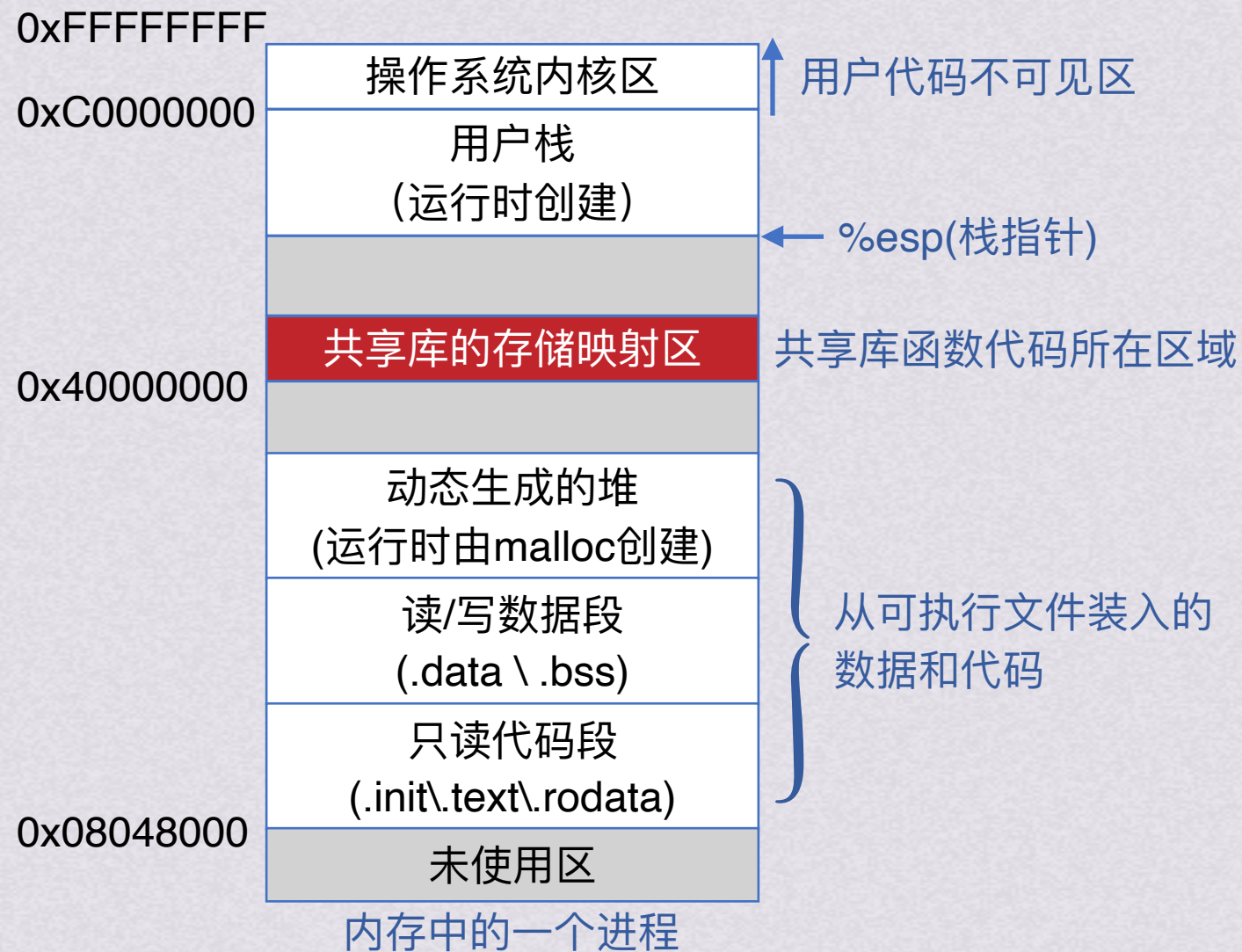
- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的内存映像

请务必学习完虚拟存储器后再观看本部分

共享库的存储映射区：将共享库（如动态链接库DLL）映射到进程的虚拟空间部分，使多个进程能共享同一个物理内存副本的库，节省了内存资源。映射区域允许进程执行库中的代码，并访问库中的数据，就像这些代码和数据是进程自己的一部分一样



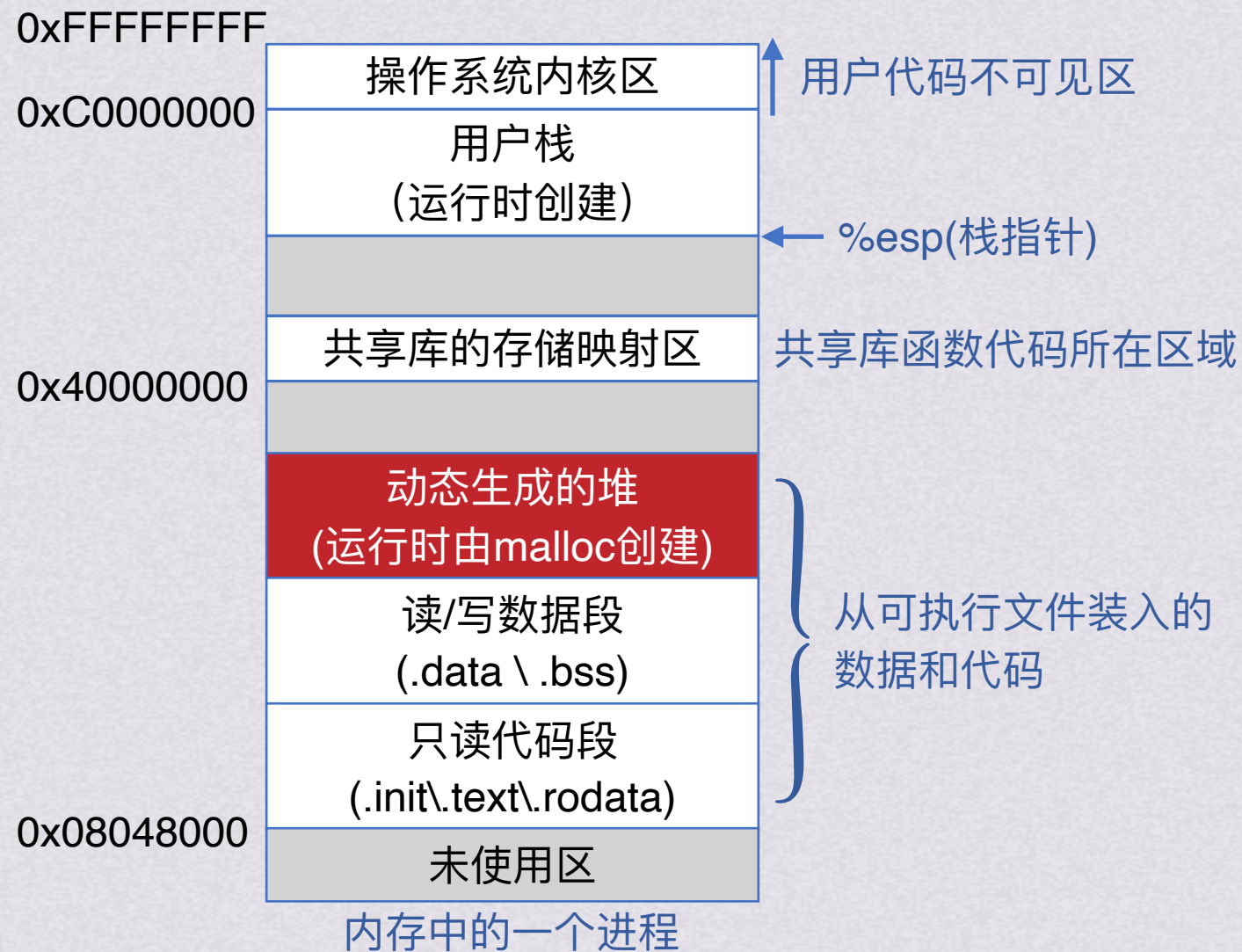
- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的内存映像

请务必学习完虚拟存储器后再观看本部分

堆：进程根据需求判断是否使用堆，使用时自行使用malloc函数申请，结束后自行使用free函数释放



进程的基本概念

进程的状态与转换

进程管理中的数据结构

进程的控制

进程通信

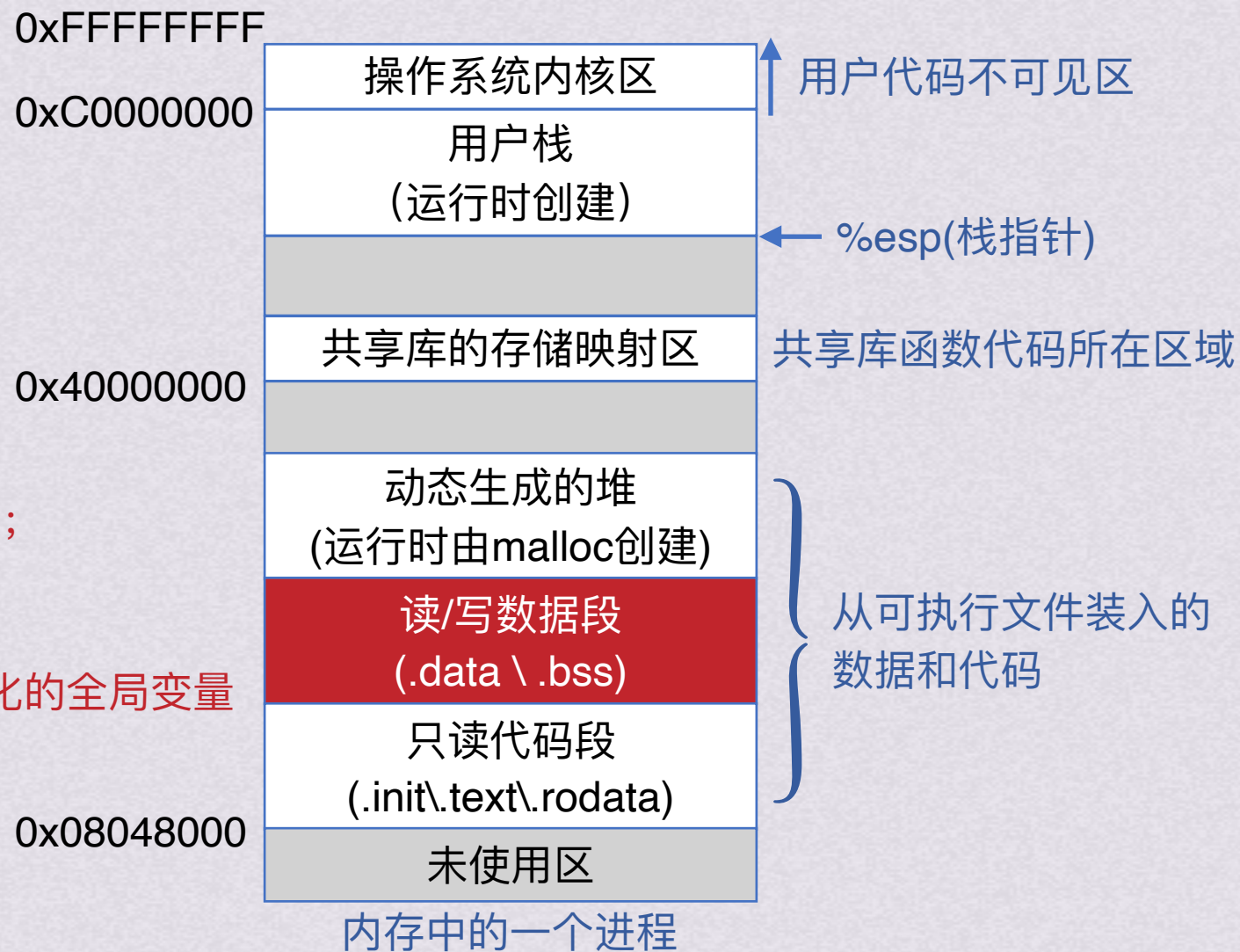


进程的内存映像

请务必学习完虚拟存储器后再观看本部分

数据段存：显式初始化为非0的全局变量；
显式初始化为非0的static局部变量

bss段存：显式初始化为0或未显式初始化的全局变量



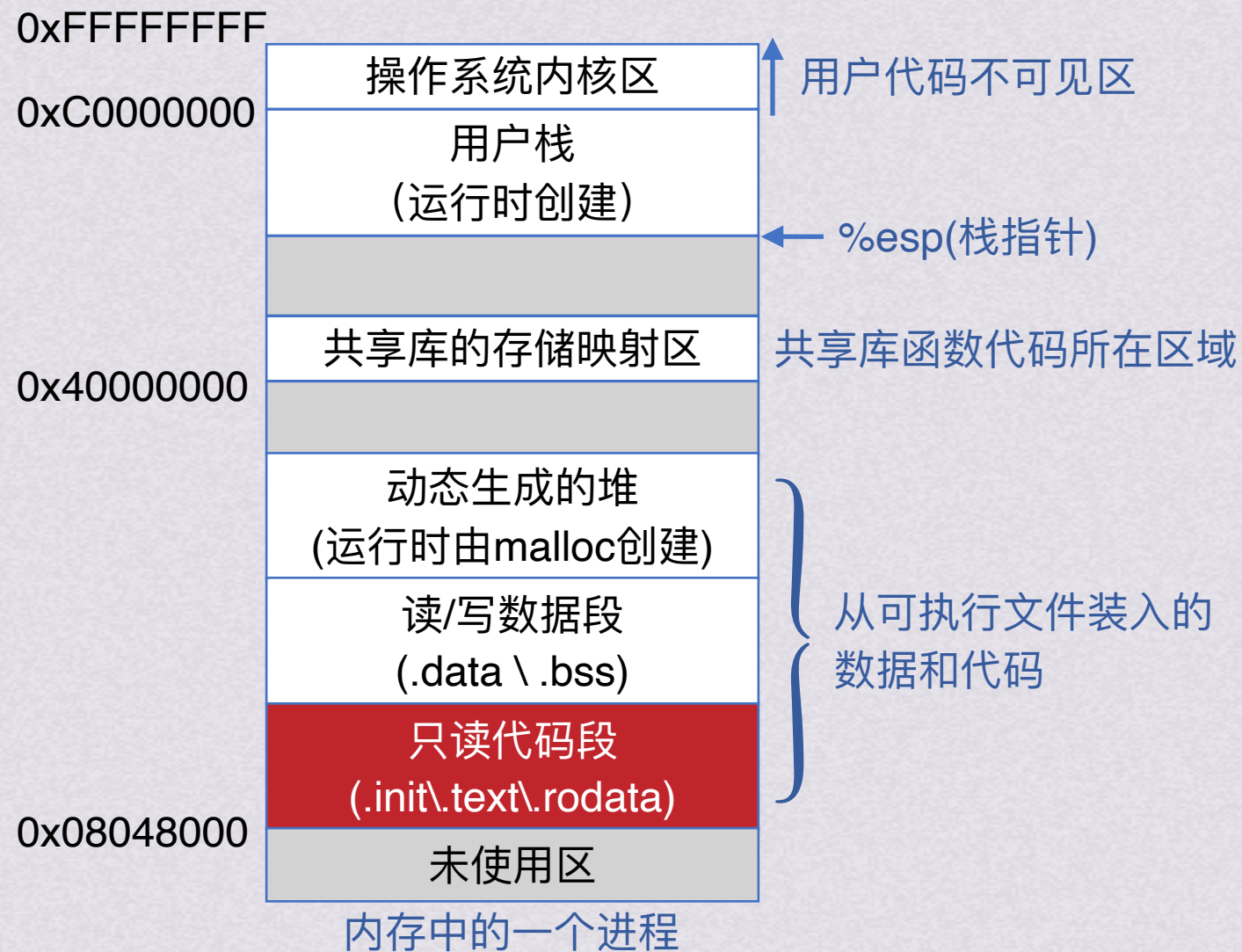
- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的内存映像

请务必学习完虚拟存储器后再观看本部分

代码段对应着程序中的代码（函数）
在linux中又叫文本段(.text)

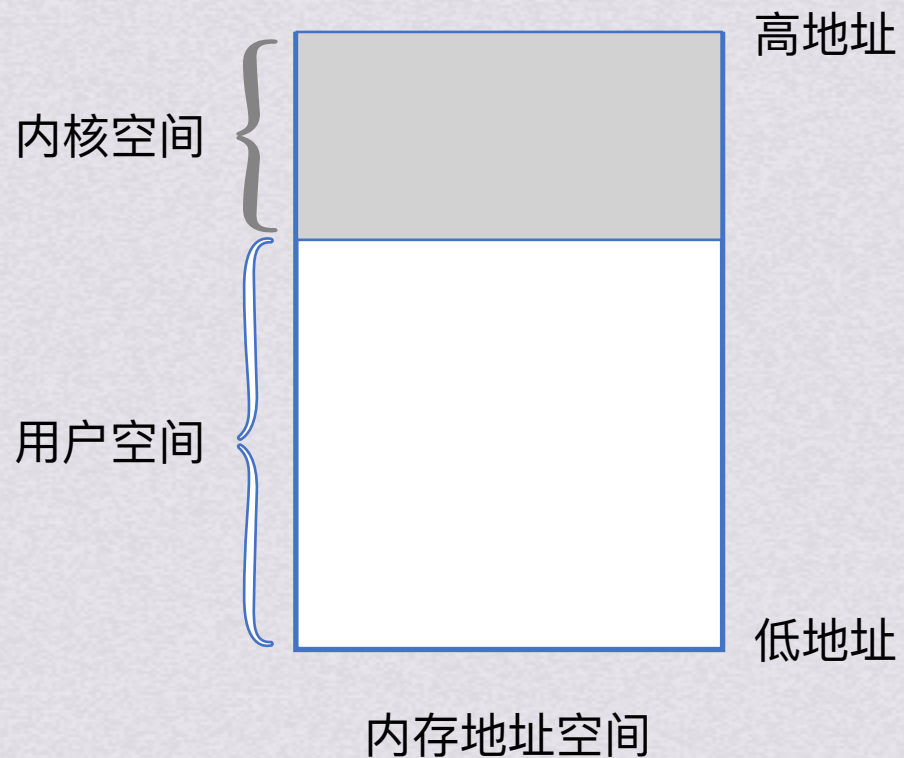


- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的内存映像

请务必学习完虚拟存储器后再观看本部分



操作系统内核区
用户栈 (运行时创建)
共享库的存储映射区
动态生成的堆 (运行时由malloc创建)
读/写数据段 (.data \ .bss)
只读代码段 (.init\ .text\ .rodata)
未使用区

用户进程的内存映像

- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的内存映像

请务必学习完虚拟存储器后再观看本部分

- 进程的基本概念
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的状态与转换

进程的基本概念

进程的内存映像

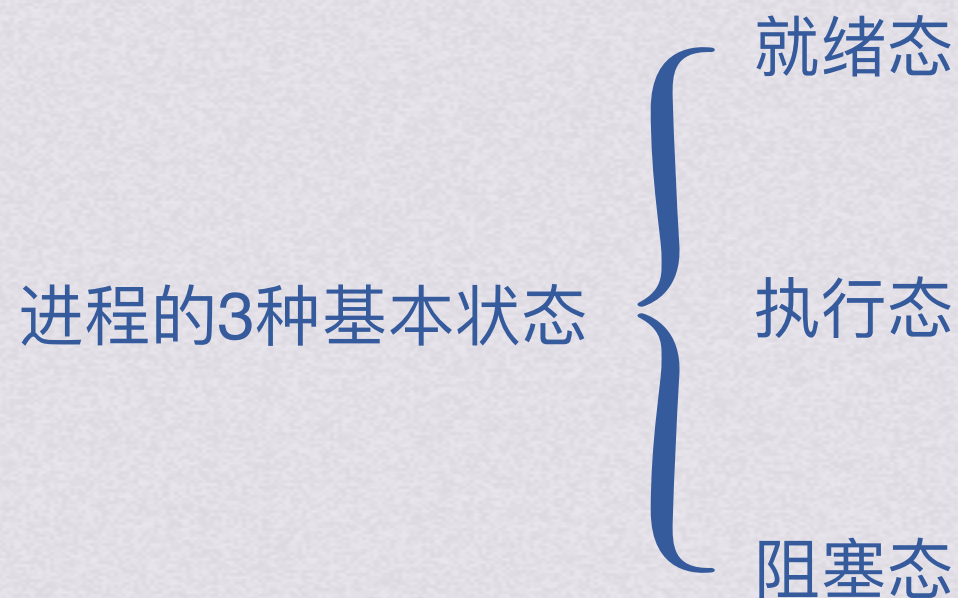
进程管理中的数据结构

进程的控制

进程通信



进程的状态与转换 基本状态与转换



- 进程的基本概念
- 进程的内存映像
- 进程管理中的数据结构
- 进程的控制
- 进程通信



进程的状态与转换 基本状态与转换

进程的3种基本状态

- 就绪态** 进程已经准备好执行，已经分配到除了CPU的所有必要资源，只要获得CPU就可以立即执行。若有多个就绪态进程，通常会将其排成队列，称就绪队列
- 执行态** 进程获得CPU后，程序“正在执行”这一状态。对任何一个时刻而言，在单处理机系统中，只有一个进程处于执行状态，在多处理机系统中，可能有多个进程处于执行状态
- 阻塞态** 正在执行的进程由于发生某事件（如io请求、申请缓冲区失败等）暂时无法执行，即进程的执行受到了阻塞。
这时会引发系统调度，OS把处理机分配给另一个就绪进程，让受阻进程处于暂停状态，这种暂停状态称为阻塞状态，处于阻塞状态的进程排成队列，叫阻塞队列

进程的基本概念

进程的内存映像

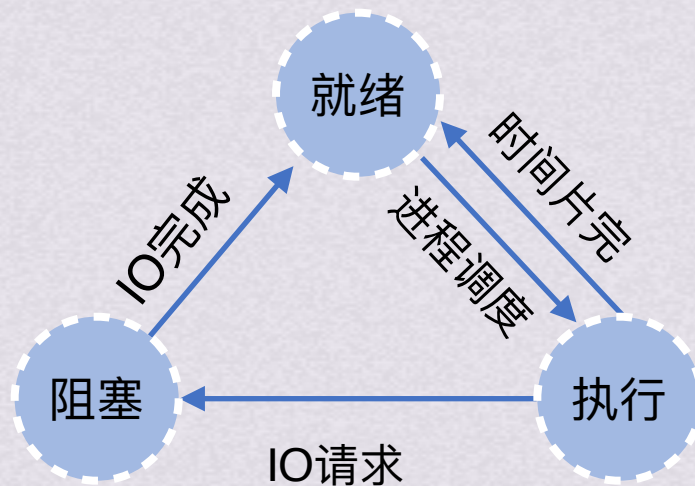
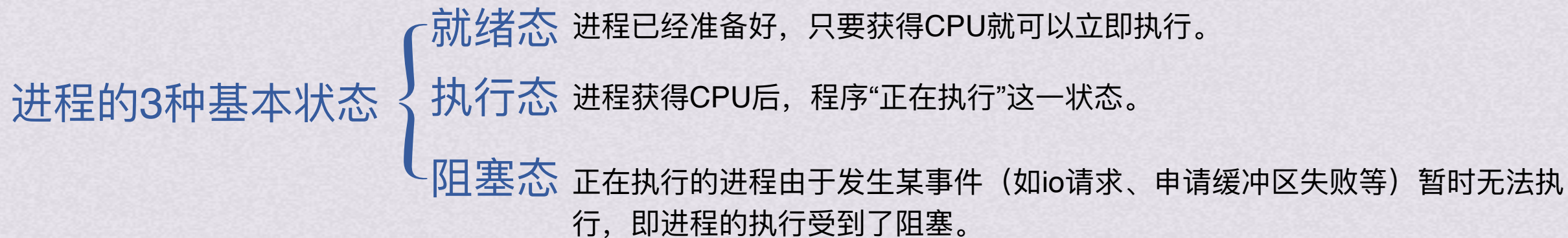
进程管理中的数据结构

进程的控制

进程通信



进程的状态与转换 基本状态与转换



进程的基本概念

进程的内存映像

进程管理中的数据结构

进程的控制

进程通信



进程的状态与转换 创建状态和终止状态

创建状态：创建进程是一个复杂的过程，一般通过多个步骤才能完成

- 1.由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
- 2.为该进程分配运行时所必需的资源
- 3.把该进程状态转换为就绪态，插入就绪队列之中

若进程所必需的资源尚不能得到满足（如内存不足），创建工作尚未完成，此时称为创建状态

进程的基本概念

进程的内存映像

进程管理中的数据结构

进程的控制

进程通信



进程的状态与转换 创建状态和终止状态

创建状态：创建进程是一个复杂的过程，一般通过多个步骤才能完成

- 1.由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
- 2.为该进程分配运行时所必需的资源
- 3.把该进程状态转换为就绪态，插入就绪队列之中

若进程所必需的资源尚不能得到满足（如内存不足），创建工作尚未完成，此时称为创建状态

两个创建进程相关的系统调用：

`fork()`:父进程调用`fork()`会**拷贝**出一个子进程，它拥有自己的地址空间、寄存器、PC等，`fork()`返回给父进程子进程的PID(**内部标识符**，后面会讲)

`exec()`:从可执行程序中加载代码和静态数据，并用它覆写自己的代码段，堆栈等其他内存空间也会被重新初始化

先`fork()`，再`exec()`，就拥有了新的进程

进程的基本概念

进程的内存映像

进程管理中的数据结构

进程的控制

进程通信



进程的状态与转换 创建状态和终止状态

创建状态：创建进程是一个复杂的过程，一般通过多个步骤才能完成

- 1.由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
- 2.为该进程分配运行时所必需的资源
- 3.把该进程状态转换为就绪态，插入就绪队列之中

若进程所必需的资源尚不能得到满足（如内存不足），创建工作尚未完成，此时称为创建状态

终止状态：进程的终止也要通过两个步骤

- 1.等待OS进行善后处理
- 2.将进程PCB清零，将PCB空间返还给OS

当一个进程到达了自然结束点，或者出现了无法克服的错误，或被OS所终止，或被其它有终止权的进程终止时，它就会进入终止状态。进入终止状态的进程不会被执行，但是OS中会保留一个记录，保存状态码和一些计时统计数据供其它进程收集

其它进程完成信息提取后，系统就会删除该进程，PCB清零并将空白PCB还给OS



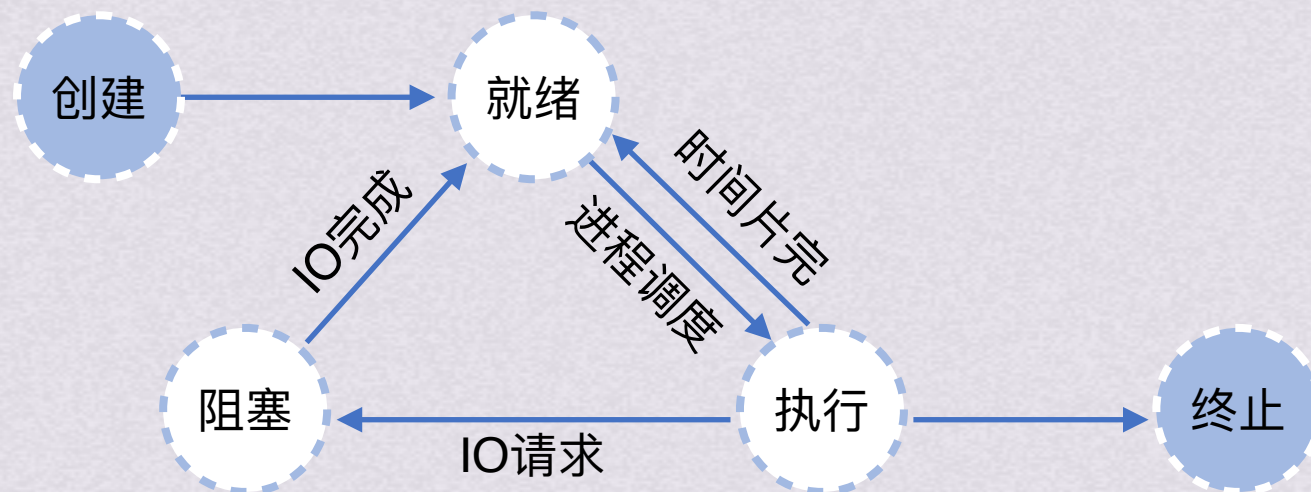
进程的状态与转换 创建状态和终止状态

创建状态：创建进程是一个复杂的过程，一般通过多个步骤才能完成

- 1.由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
- 2.为该进程分配运行时所必需的资源
- 3.把该进程状态转换为就绪态，插入就绪队列之中

终止状态：进程的终止也要通过两个步骤

- 1.等待OS进行善后处理
- 2.将进程PCB清零，将PCB空间返还给OS。

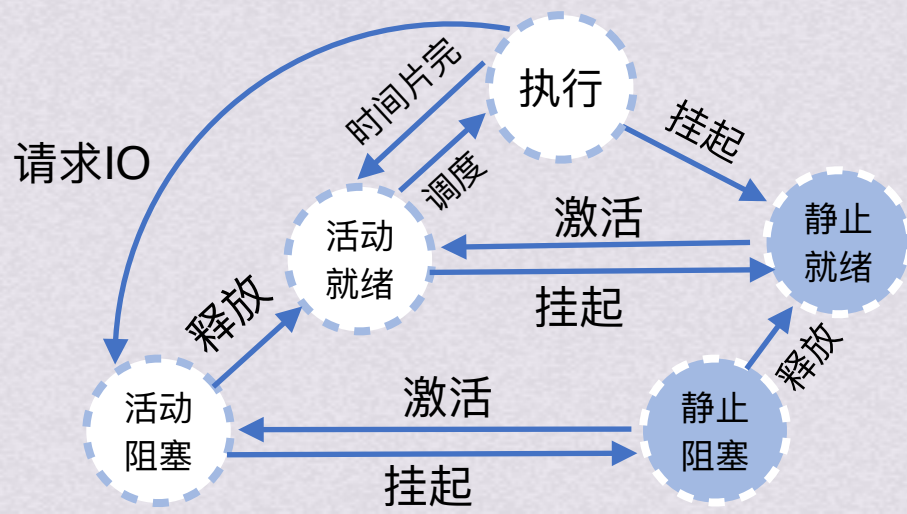




进程的状态与转换 挂起操作和进程状态的转换

在很多系统中，除去就绪、执行、阻塞以外还引入了一个“挂起”操作，与之对应的是“激活”操作
有以下几种状态转换

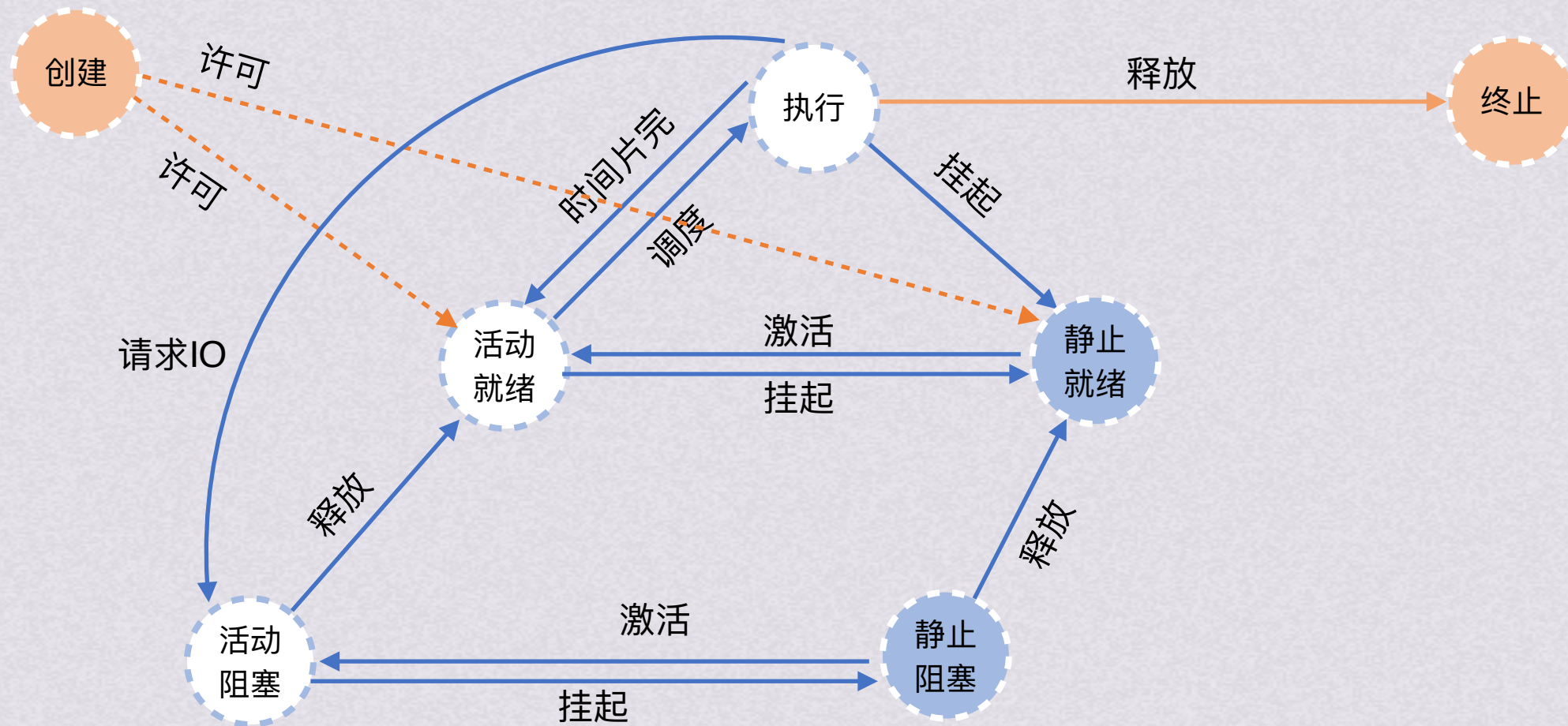
- 1.活动就绪->静止就绪**:进程处于未被挂起的就绪状态，称为活动就绪状态，此时进程可以接受调度。当用挂起原语Suspend将进程挂起后，该进程的状态转换为静止就绪状态，并且不会再被调度执行
- 2.活动阻塞->静止阻塞**:进程处于未被挂起的阻塞态，称活动阻塞态，使用挂起原语Suspend将进程挂起后，进程的状态转换成了静止阻塞状态，这样状态的进程在所期待的事件发生后，从静止阻塞状态转变为静止就绪状态
- 3.静止就绪->活动就绪**:静止就绪的进程，使用激活原语Active将其激活，则该进程的状态会转换为活动就绪状态，得到CPU资源就可执行
- 4.静止阻塞->活动阻塞** 静止阻塞的进程，使用激活原语Active将其激活，该进程的状态会转换为活动阻塞状态





进程的状态与转换 挂起操作和进程状态的转换

在很多系统中，除去就绪、执行、阻塞以外还引入了一个“挂起”操作，与之对应的是“激活”操作

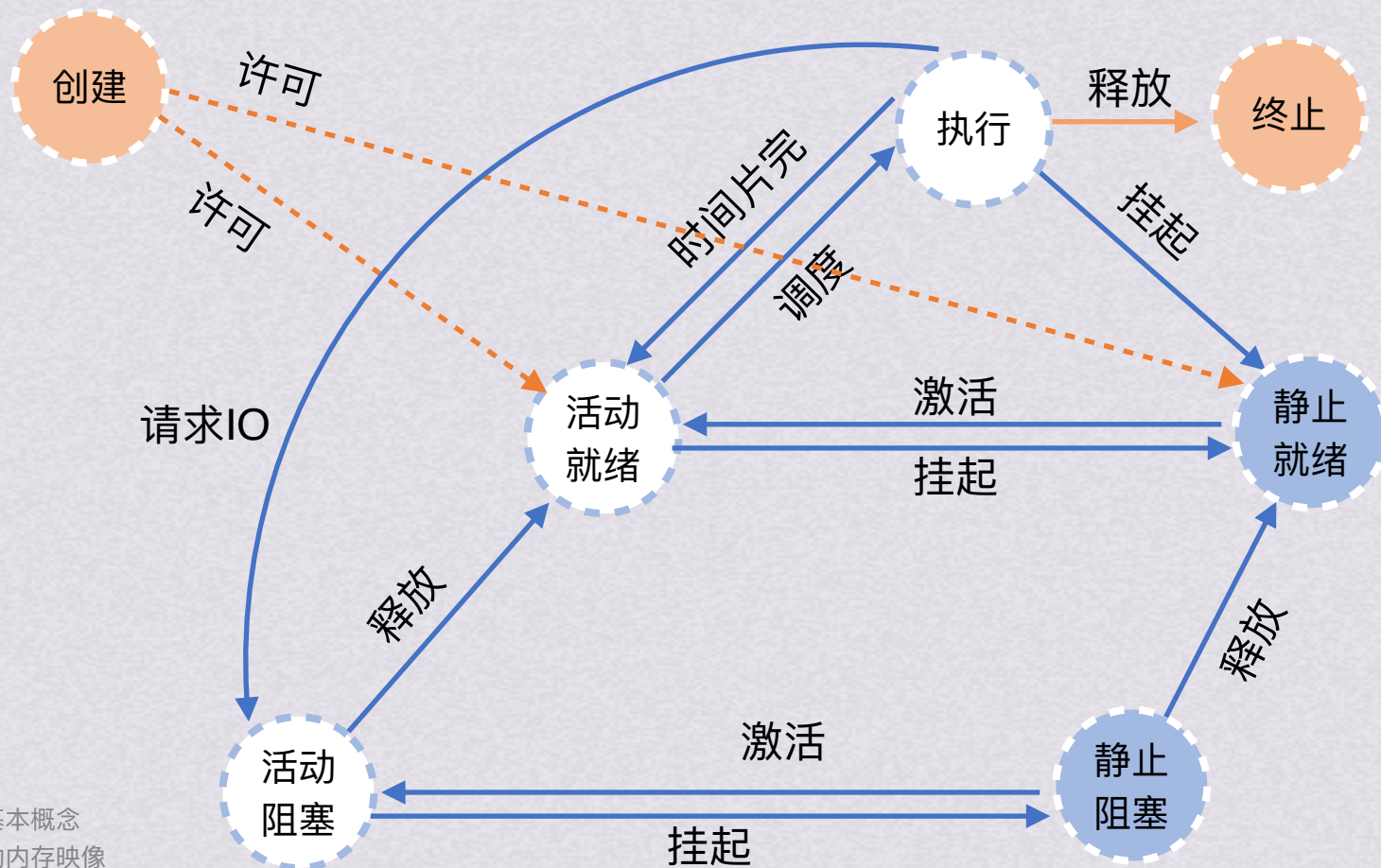


具有创建、终止和挂起状态的进程状态图



进程的状态与转换 挂起操作和进程状态的转换

在很多系统中，除去就绪、执行、阻塞以外还引入了一个“挂起”操作，与之对应的是“激活”操作



比起之前5个状态转换，需要额外考虑以下情况

- 1.NULL->创建：一个新进程产生时处于创建状态
- 2.创建->活动就绪：在当前系统性能和内存容量允许的情况下，完成进程创建必要操作后，系统会将进程状态转换为活动就绪状态
- 3.创建->静止就绪：当前的系统资源状况和性能要求不允许的情况下，系统不会分配给新建进程所需资源（主要是内存），还会将进程转换为静止就绪状态。进程放在外存不参与调度，创建工作尚未完成
- 4.执行->终止：一个进程已完成任务，或出现了无法客服的错误，或被OS或其它进程所终止时，将进程状态转换为终止状态

具有创建、终止和挂起状态的进程状态图



进程的状态与转换

进程的基本概念

进程的内存映像

进程管理中的数据结构

进程的控制

进程通信



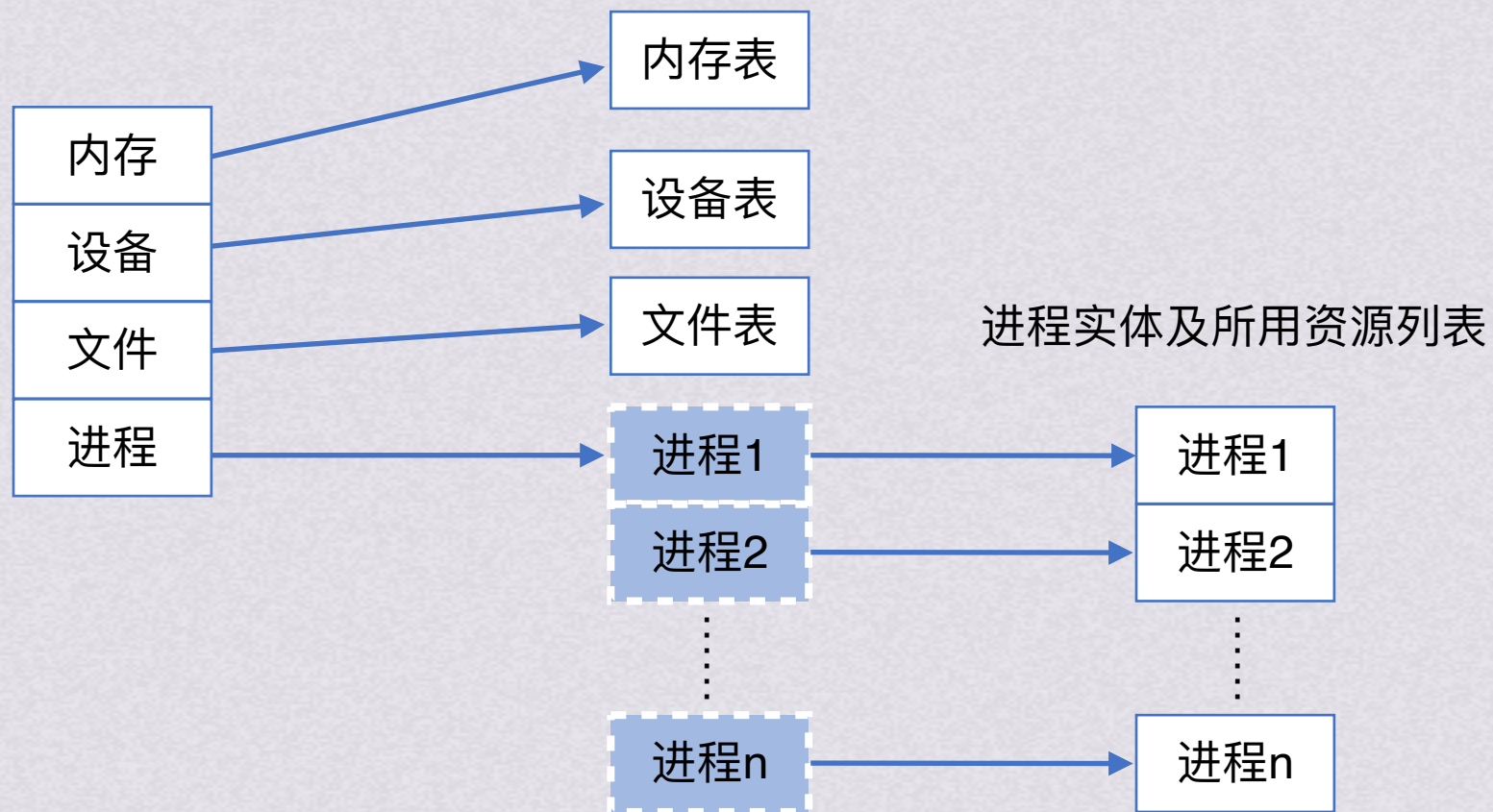
进程管理中的数据结构

- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程的控制
- 进程通信



进程管理中的数据结构

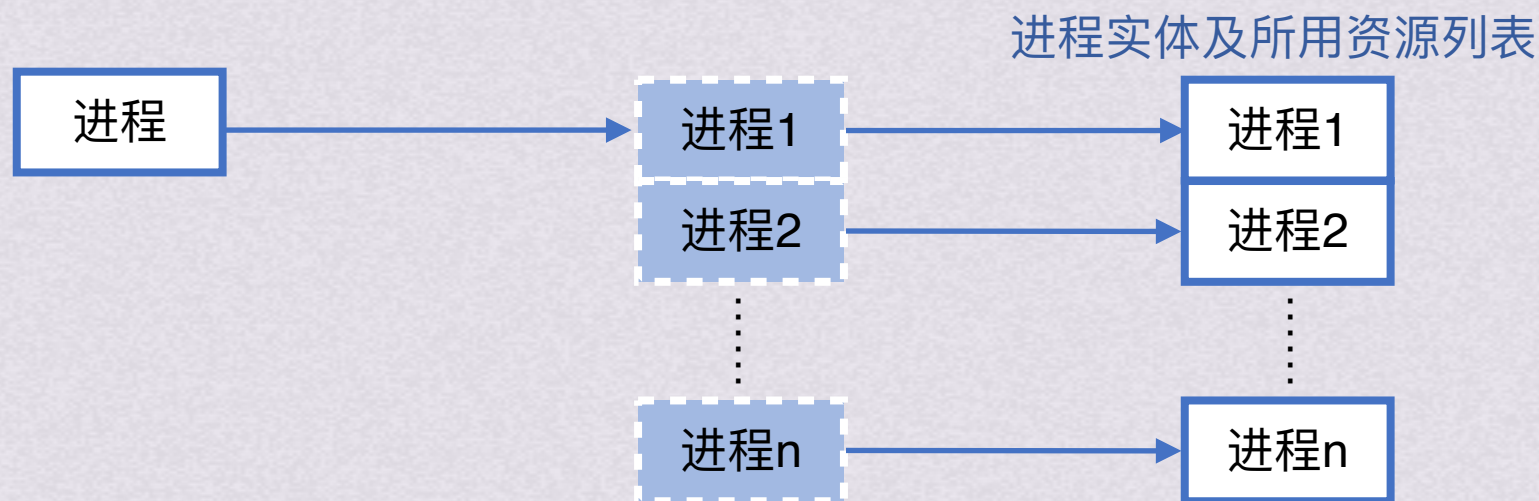
为了便于管理和使用计算机中的各类资源，OS将它们抽象为相应的各种数据结构，并提供一组对资源进行操作的命令





进程管理中的数据结构

为了便于管理和使用计算机中的各类资源，OS将它们抽象为相应的各种数据结构，并提供一组对资源进行操作的命令



为了便于系统描述和管理进程，OS为每个进程定义了一个数据结构**PCB(进程控制块)**。PCB用于记录OS所需的、用于描述进程当前情况以及管理进程运行状态的全部信息，是OS中最重要的记录型数据结构。



进程管理中的数据结构

为了便于管理和使用计算机中的各类资源，OS将它们抽象为相应的各种数据结构，并提供一组对资源进行操作的命令

为了便于系统描述和管理进程，OS为每个进程定义了一个数据结构**PCB(进程控制块)**
PCB用于记录OS所需的、用于描述进程当前情况以及管理进程运行状态的全部信息
是OS中最重要的记录型数据结构

- PCB具体作用：**
- 1.作为独立运行基本单位的标志**，一个程序（含数据）配置了PCB后，就表示它已经是一个能在多道程序环境下独立运行的基本单位了。创建新进程时会建立一个PCB；进程结束时会回收其PCB，进程也随之消亡。OS通过PCB来感知进程的存在，也是进程存在于系统中的唯一标志
 - 2.实现间断性运行方式**，多道程序环境下，程序是停停走走的，进程被阻塞的时候必须保留自己运行时CPU的现场信息，再次被调度运行时需要恢复CPU现场信息。有了PCB，系统就可以把这些信息保存在其中
 - 3.提供进程管理所需要的信息**，当调度程序调度到某进程时，只能根据该进程PCB中记录的程序和数据在内存/外存的起始地址中找到相应程序/数据。进程需要访问文件或者IO时，也需要借助PCB中信息，还可根据PCB了解该进程所需全部资源等信息
 - 4.提供进程调度所需信息**，PCB中有进程所处状态等信息，也会根据需要记录优先级、等待时间、已执行时间等信息
 - 5.实现与其它进程的同步和通信**，PCB中还有用于实现进程通信的区域或通信队列的指针等

进程的基本概念

进程的内存映像

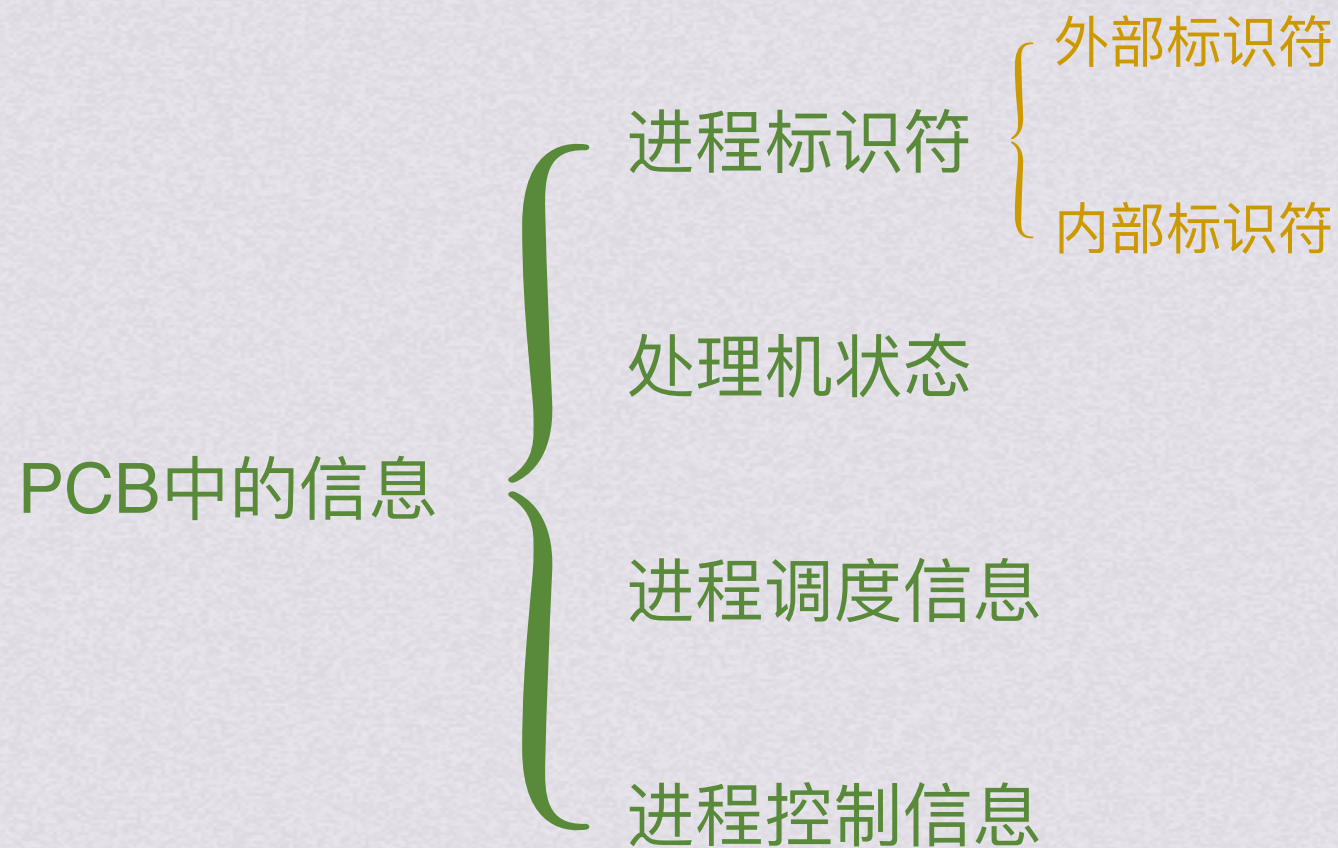
进程的状态与转换

进程的控制

进程通信



进程管理中的数据结构



- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程的控制
- 进程通信



进程管理中的数据结构 PCB中的信息

进程标识符

外部标识符 为方便用户对进程的访问，为每个进程设置一个外部标识符。通常由字母和数字组成。为了描述进程家族关系，还应设置父进程标识符和子进程标识符。此外还可设置用户标识符，指示拥有该进程的用户

内部标识符 为方便系统对进程的使用，在OS中又为进程设置了内部标识符，赋予每个进程唯一一个数字标识符，通常是一个进程序号

- 处理机状态
- 进程调度信息
- 进程控制信息
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程的控制
- 进程通信



进程管理中的数据结构 PCB中的信息

处理机状态

处理机的状态信息，也称处理机的上下文，主要由处理机的各种寄存器中的内容组成，包括：

- 1.通用寄存器(用户可视寄存器)：可被用户程序访问，用于暂存信息
- 2.指令计数器：其中存放了要访问下一条指令的地址
- 3.程序状态字寄存器：每个用户进程都有与之相关的系统栈，用于存放进程和系统的调用参数和调用地址。栈指针指向该栈栈顶。处理机执行状态时，正在处理的许多信息也是放在寄存器中的。进程被切换时，处理机状态信息都必须保存在相应PCB中
- 4.堆栈指针寄存器：指向用户自己堆栈的指针和指向内核栈的指针

{ 进程标识符
进程调度信息
进程控制信息
进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



进程管理中的数据结构 PCB中的信息

进程调度信息

因为OS要对进程进行调度，所以需要了解必要的信息，包括以下信息

- 1.进程状态：指明进程当前的状态(阻塞、就绪、运行)，作为进程调度、对换的依据
- 2.进程优先级：描述进程使用处理机的优先级别（用一个整数表示），优先级高的优先获得处理机
- 3.进程调度所需的其它信息：部分进程调度算法需要知道进程已等待CPU时间总和、进程已执行时间总和等
- 4.事件：进程由执行状态转换为阻塞状态所等待发生的事件（阻塞的原因）

{ 进程标识符
处理机状态
进程控制信息
进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



进程管理中的数据结构 PCB中的信息

进程控制信息

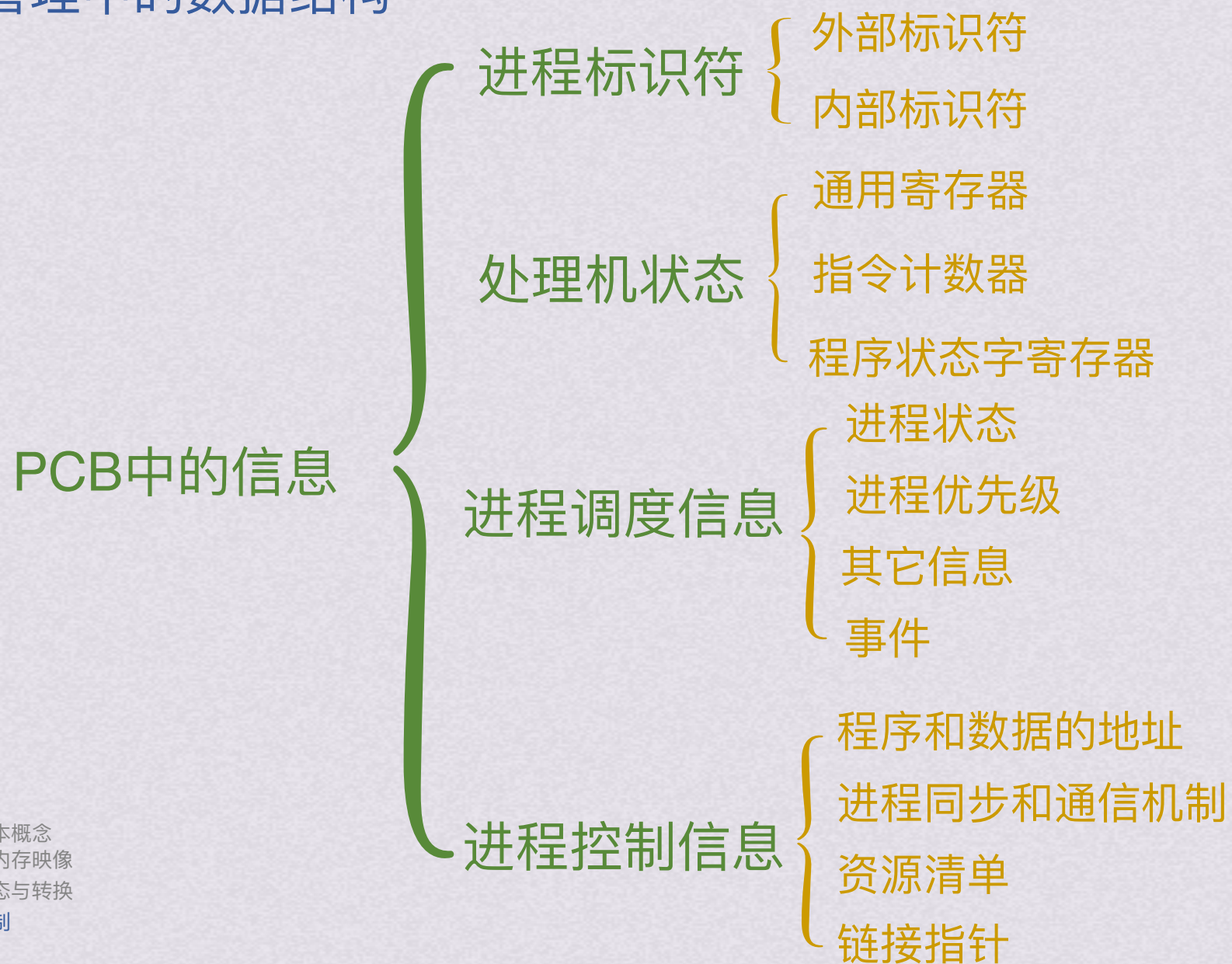
指用于进程控制所必需的信息

- 1.程序和数据的地址：即程序和数据的内存或外存起始地址，便于该进程执行时能从PCB中快速找到程序和数据
- 2.进程同步和通信机制：比如消息队列指针、信号量等，它们可能会全部或部分放在PCB中
- 3.资源清单：列出了该进程在运行期间所需的全部资源（除CPU外）
- 4.链接指针：给出本进程所在队列中的下一个进程的PCB起始地址

{ 进程标识符
处理机状态
进程调度信息
进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



进程管理中的数据结构



进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



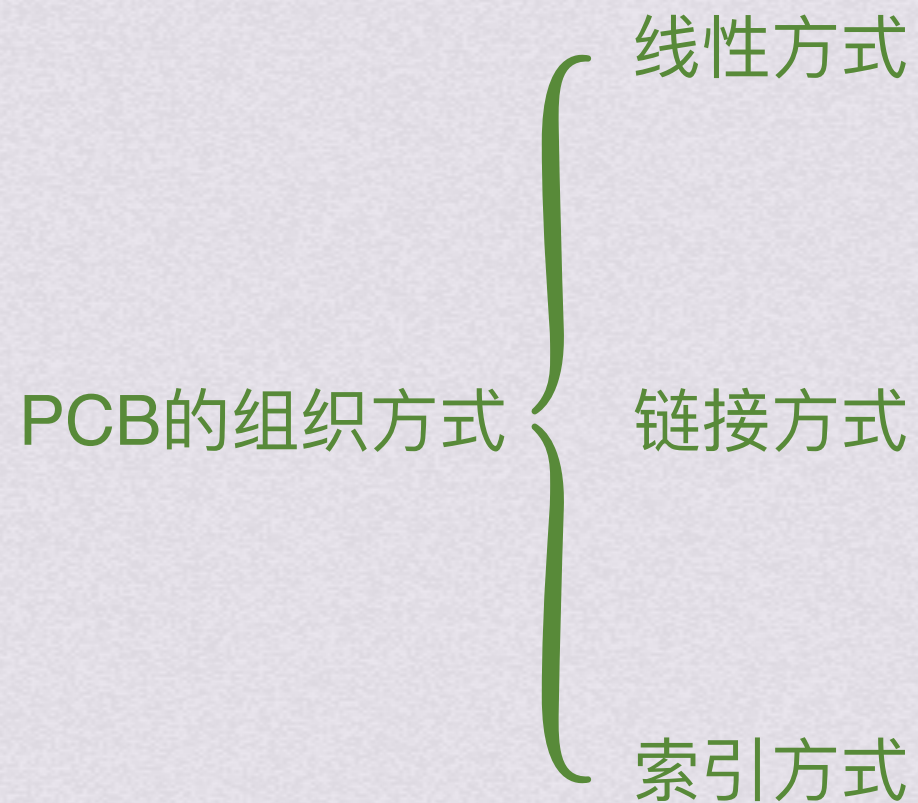
进程管理中的数据结构

PCB的组织方式

进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



进程管理中的数据结构

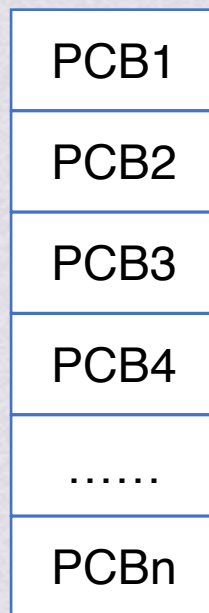


- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程的控制
- 进程通信



进程管理中的数据结构 PCB的组织方式

线性方式

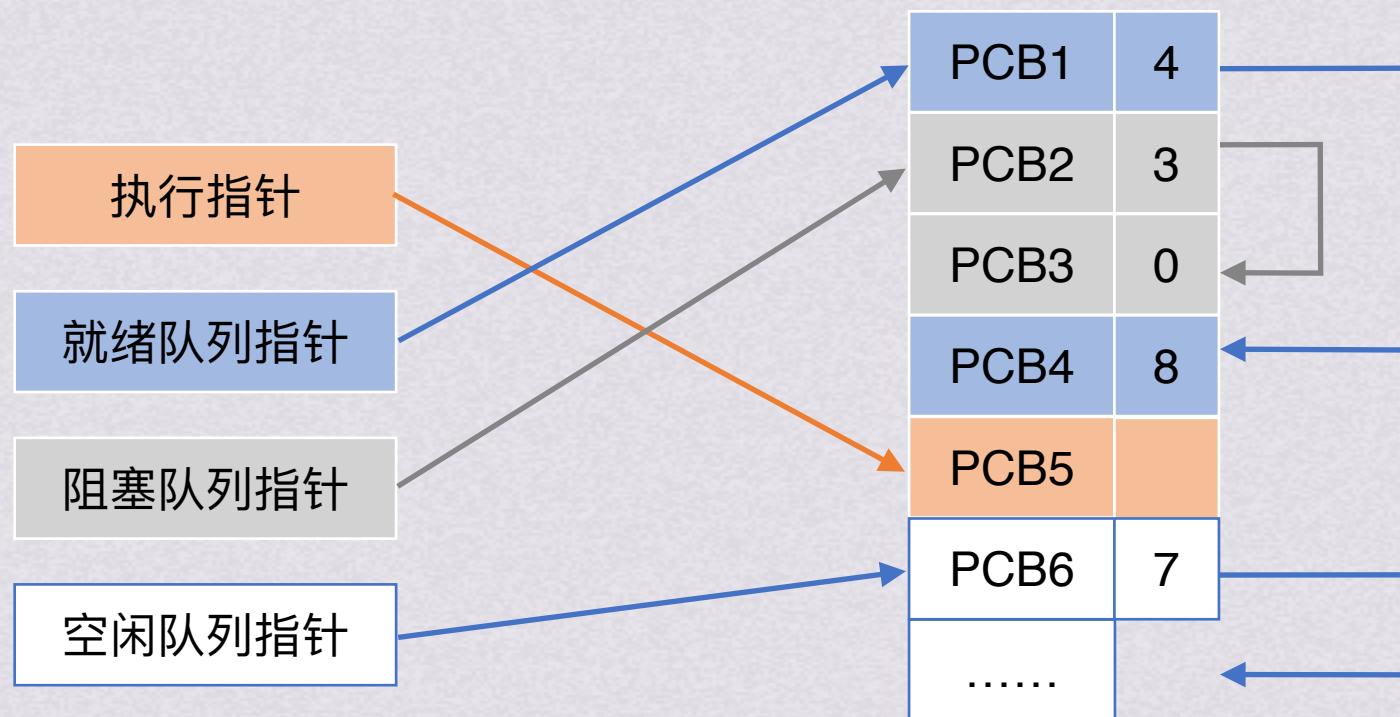


所有PCB组织在一张线性表中，起始地址放在内存的一个专用区域中
这样的方式简单开销小，但每次查找都需要扫描整张表，适合进程不多的系统



进程管理中的数据结构 PCB的组织方式

链接方式



将具有相同状态的进程PCB分别链接成一个队列

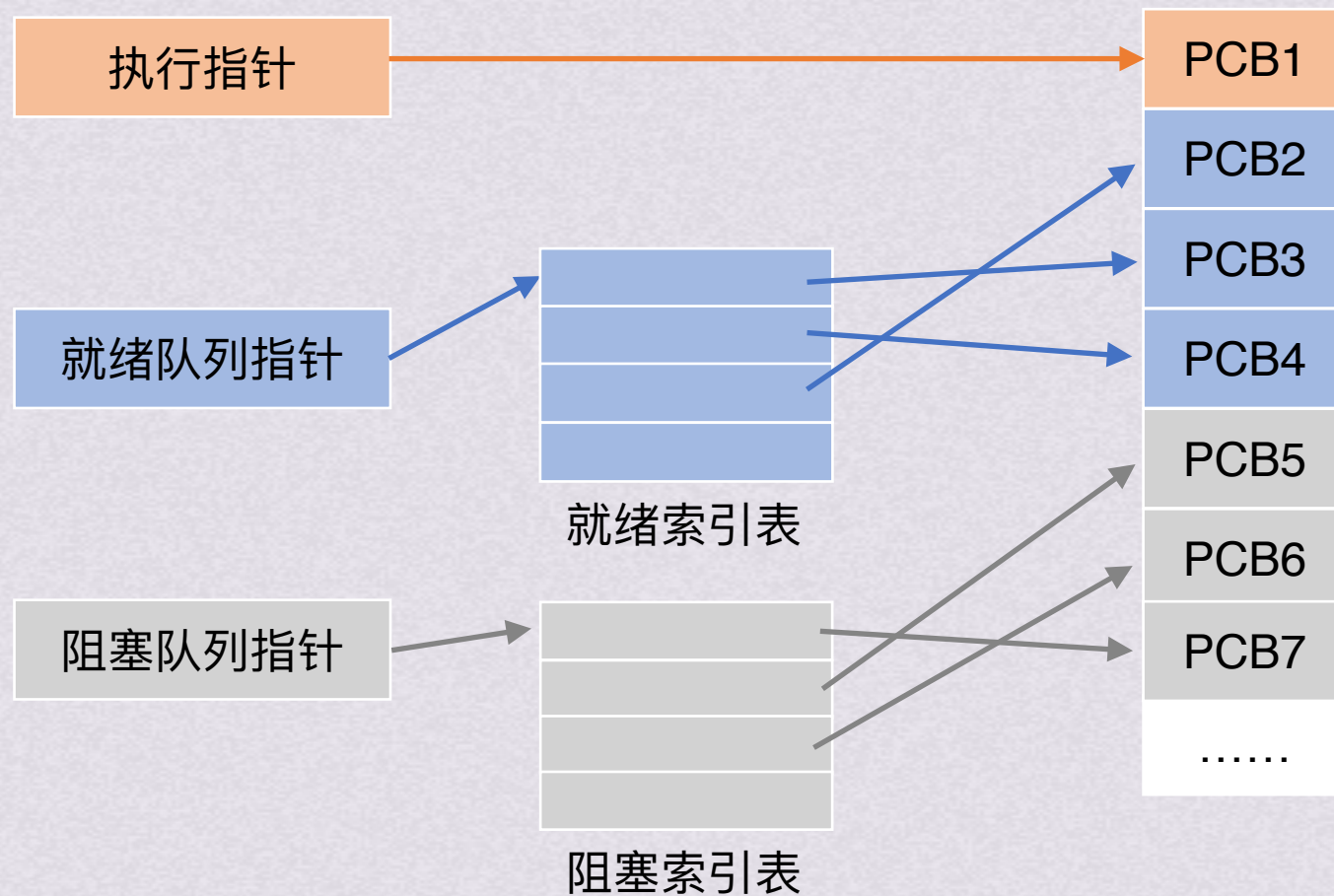
就绪队列可按照优先级排序

阻塞队列可以根据阻塞原因分成多个阻塞队列



进程管理中的数据结构 PCB的组织方式

索引方式



OS根据进程状态的不同，建立几张索引表，把各索引表在内存中起始地址记录在内存的专用单元中
每个索引表的表目中，记录具有相应状态的某个PCB在PCB表中的地址



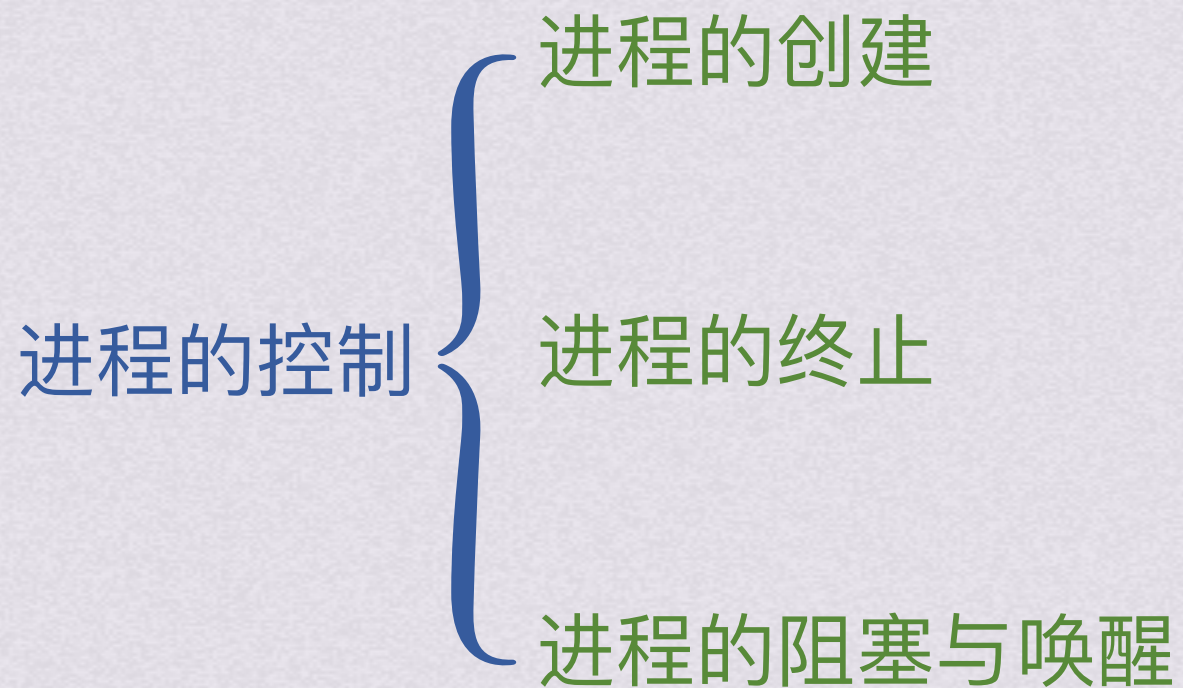
进程管理中的数据结构

进程的基本概念
进程的内存映像
进程的状态与转换
进程的控制
进程通信



进程的控制

- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



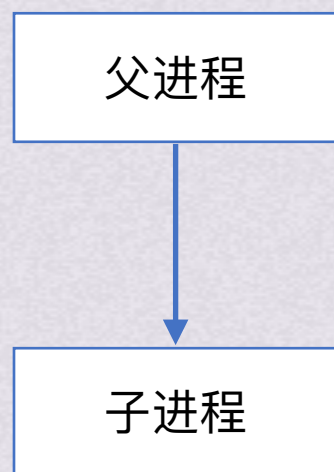
- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的创建

os中允许一个进程创建另一个进程，通常把创建进程的进程称为父进程，被创建的称为子进程。子进程可以继续创建其自己的子进程（即父进程的孙进程）。如在UNIX系统中，进程与其子孙进程可以共同组成一个进程家族（进程组）



子进程可以继承父进程所拥有的资源(打开的文件，分配的缓冲区)。当子进程被撤销时，应将从父进程那里获得的资源归还父进程。

在撤销父进程时，也必须同时撤销所有的子进程

在windows系统中不存在任何进程层次结构的概念，所有进程有相同的地位。一个进程创建了另外一个进程后，创建进程获得一个句柄，作用相当于令牌，可控制被创建进程，同时句柄可以进行传递。

因此进程间关系不是层次关系，而是控制与被控制的简单关系



进程的控制

引起进程创建的事件

用户登陆

分时系统中，用户在终端输入登录命令后，若登录成功，则系统会为该用户创建一个进程，并插入就绪队列

作业调度

多道批处理系统中，作业调度程序按一定的算法调度到某个队列中

提供服务

运行中的用户程序提出某种请求后，系统专门创建一个进程为用户提供所需服务（如打印文件创建打印进程）

应用请求

上面三种情况是OS内核为用户创建新进程；针对应用请求则是用户进程自己创建新进程，进程间并发执行完成任务

进程的终止

进程的阻塞与唤醒

进程的基本概念

进程的内存映像

进程的状态与转换

进程管理中的数据结构

进程通信



进程的控制

进程的创建

进程创建的步骤：1.申请空白PCB，为新进程申请一个唯一的数字标识符

2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间

- 进程的终止
- 进程的阻塞与唤醒
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的创建

进程创建的步骤：1.申请空白PCB，为新进程申请一个唯一的数字标识符

2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间

- 进程的终止
- 进程的阻塞与唤醒
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信

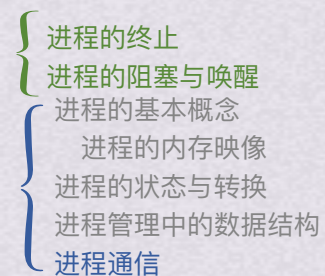


进程的控制

进程的创建

进程创建的步骤：

- 1.申请空白PCB，为新进程申请一个唯一的数字标识符
- 2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
- 3.初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；



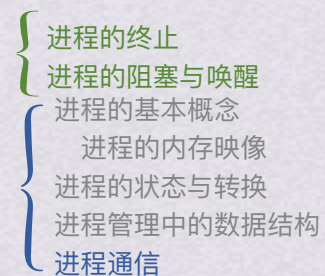


进程的控制

进程的创建

进程创建的步骤：

- 1.申请空白PCB，为新进程申请一个唯一的数字标识符
- 2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
- 3.初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；
 - 3-2 初始化处理机状态信息，使程序计数器指向程序的入口地址，使栈指针指向栈顶
 - 3-3 初始化处理机控制信息，将进程的状态设置为就绪状态或静止就绪状态，通常还须将其设置为最低优先级



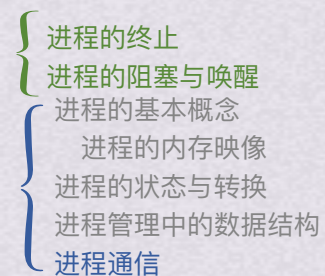


进程的控制

进程的创建

进程创建的步骤：

- 1.申请空白PCB，为新进程申请一个唯一的数字标识符
- 2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
- 3.初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；
 - 3-2 初始化处理机状态信息，使程序计数器指向程序的入口地址，使栈指针指向栈顶
 - 3-3 初始化处理机控制信息，将进程的状态设置为就绪状态或静止就绪状态，通常还须将其设置为最低优先级
- 4.如果进程就绪队列能够接纳新进程，就将新进程插入就绪队列



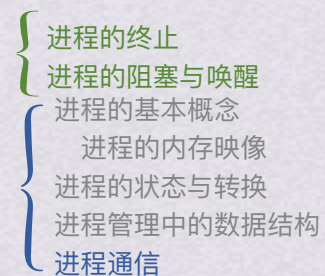


进程的控制

进程的创建

进程创建的步骤：

- 1.申请空白PCB，为新进程申请一个唯一的数字标识符
- 2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
- 3.初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；
 - 3-2 初始化处理机状态信息，使程序计数器指向程序的入口地址，使栈指针指向栈顶
 - 3-3 初始化处理机控制信息，将进程的状态设置为就绪状态或静止就绪状态，通常还须将其设置为最低优先级
- 4.如果进程就绪队列能够接纳新进程，就将新进程插入就绪队列



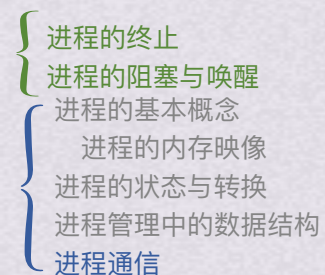


进程的控制

进程的创建

进程创建的步骤：

- 1.申请空白PCB，为新进程申请一个唯一的数字标识符
- 2.为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
- 3.初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；
 - 3-2 初始化处理机状态信息，使程序计数器指向程序的入口地址，使栈指针指向栈顶
 - 3-3 初始化处理机控制信息，将进程的状态设置为就绪状态或静止就绪状态，通常还须将其设置为最低优先级
- 4.如果进程就绪队列能够接纳新进程，就将新进程插入就绪队列





进程的控制

进程的创建

进程创建新进程时，有两种执行的可能：

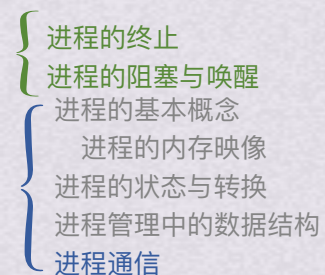
父进程与子进程并发执行

父进程等待，直到其某个或全部子进程执行完毕

新进程的内存映像也有两种可能

子进程是父进程的复制品（子进程与父进程具有相同程序和数据）

子进程加载另一个新程序





进程的控制

进程的创建

- 进程的终止
- 进程的阻塞与唤醒
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的终止

- 进程的创建
- 进程的阻塞与唤醒
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的终止

正常结束

进程的任务已完成，准备退出运行。任何系统中都应有一个用于表示进程已经运行完成的指示。通常会在程序的最后安排一条指令，用于向OS表示运行已结束。程序运行到该指令时，会产生一个中断通知OS本进程运行完毕

异常结束

指进程在运行时，发生了某种异常事件，使程序无法继续运行。

常见的异常事件有：①越界错，程序所访问的存储区已越出该进程所占存储区域的范围；

②保护错，进程试图去访问一个不允许访问的资源或文件，或者以不适当的方式进行访问，例如，进程试图去写一个只读文件；

③指令错，程序试图去执行一条不存在的指令(非法指令)，出现该错吴的原因可能是程序错误地转移到了数据区，把数据当成了指令；

④特权指令错，进程试图去执行一条只允许OS执行的指令；

⑤运行超时，进程的运行时间超过了设定的最大值；

⑥等待超时，进程等待某事件的时间超过了指定的最大值；

⑦算术运算错，进程试图去执行一个被禁止的运算，例如，被0除；

⑧IO错，这是指在IO过程中发生了错误等。

外界干预

进程应外接请求而终止（操作员或OS干预，如死锁）

引起进程终止的操作

进程的创建

进程的阻塞与唤醒

进程的基本概念

进程的内存映像

进程的状态与转换

进程管理中的数据结构

进程通信



进程的控制

进程的终止

进程终止的过程

发生了要求终止进程的事件后，OS会调用进程终止原语，按照一定步骤终止指定进程

- 1.根据被终止进程的标识符，从PCB集合中找到该进程PCB，读出该进程状态
- 2.若被终止进程处于执行状态，立刻中止执行
- 3.若该进程还有子孙进程，还应终止其所有子孙进程，以防它们称为不可控的进程
- 4.将被终止进程拥有的全部资源归还给父进程或系统
- 5.将被终止进程的PCB从所在队列中移除

进程的创建

进程的阻塞与唤醒

进程的基本概念

进程的内存映像

进程的状态与转换

进程管理中的数据结构

进程通信



进程的控制

进程的终止

- 进程的创建
- 进程的阻塞与唤醒
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的阻塞与唤醒

- 进程的创建
- 进程的终止
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的阻塞与唤醒

- 进程的创建
- 进程的终止
- 进程的基本概念
 - 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



进程的控制

进程的阻塞与唤醒

引起进程阻塞的事件

向系统请求共享资源失败

(请求打印机，但是已经没可分配打印机了)

等待某种操作的完成

(请求IO设备，但要拿到IO设备进行IO操作的结果才能继续执行)

新数据尚未到达

(对于合作进程，一个进程需要另一进程的数据才能处理，而该数据尚未到达)

等待新任务的到达

(某些OS中会设置特定系统进程，完成任务后自我阻塞以等待新任务到来)

进程的创建
进程的终止

进程的基本概念
进程的内存映像
进程的状态与转换
进程管理中的数据结构
进程通信



进程的控制

进程的阻塞与唤醒

进程阻塞的过程

正在执行的进程，发生引起阻塞的事件，就会调用阻塞原语block将自己阻塞，故阻塞是进程自身的主动行为。进入block阶段后，由于该进程还处在执行状态，OS停止执行该进程，PCB中改为阻塞态，将PCB插入阻塞队列，再转至调度程序进行重新调度操作。

进程唤醒的过程

被阻塞进程所期待的事件发生时，就会调用唤醒原语wakeup以将等待该事件的进程唤醒。

wakeup的执行过程：

1. 将被阻塞的进程从等待该事件的阻塞队列中移出，将其PCB中的现行状态改为就绪。
2. 将该PCB插入就绪队列。

阻塞原语和唤醒原语是一对作用相反的原语，使用它们时必须成对使用。



进程的控制

- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程通信



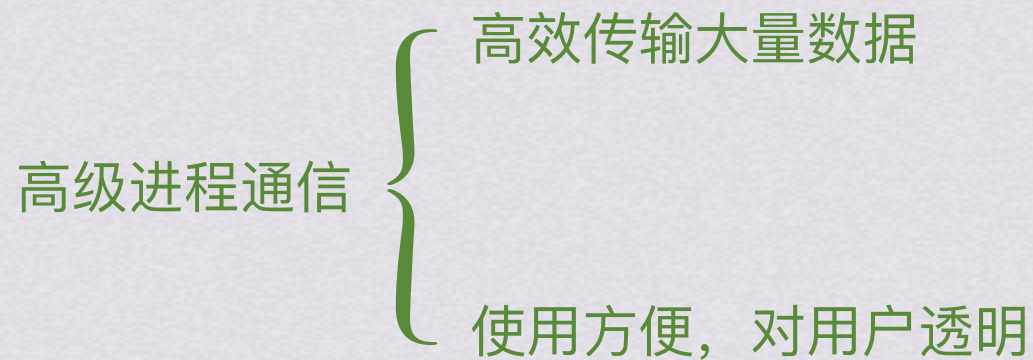
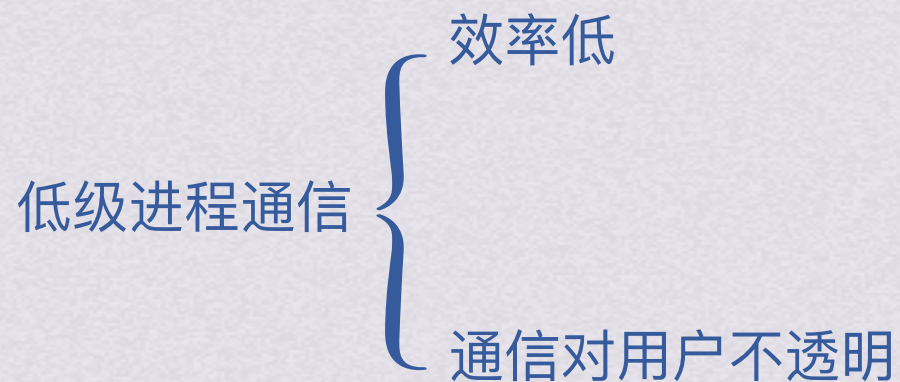
进程通信

- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制

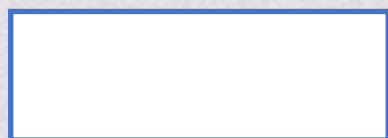


进程通信

进程通信有低级、高级之分



会在后续章节中重点介绍！



缓冲区

{

- 进程的基本概念
- 进程的内存映像
- 进程的状态与转换
- 进程管理中的数据结构
- 进程的控制



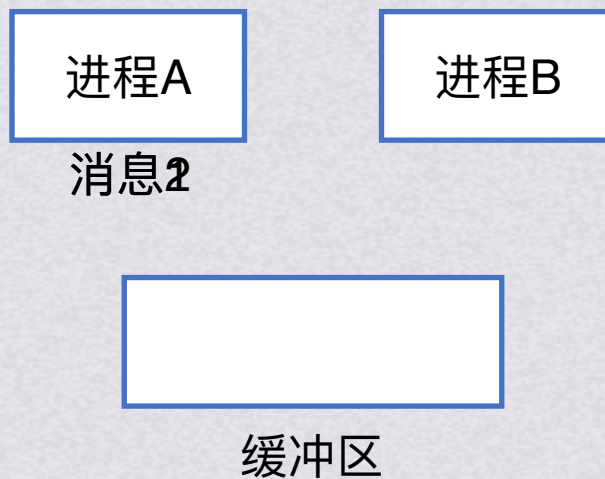
进程通信

共享存储器系统

共享存储器系统中，相互通信的进程共享某些数据结构或存储区，进程之间通过这些空间进行通信

1. 基于共享数据结构的通信方式

要求各进程共享某些数据结构完成信息交换，OS仅提供共享存储器，由程序员负责对共享数据结构进行设置和对进程间同步进行处理。效率低下，属于低级进程通信





进程通信

共享存储器系统

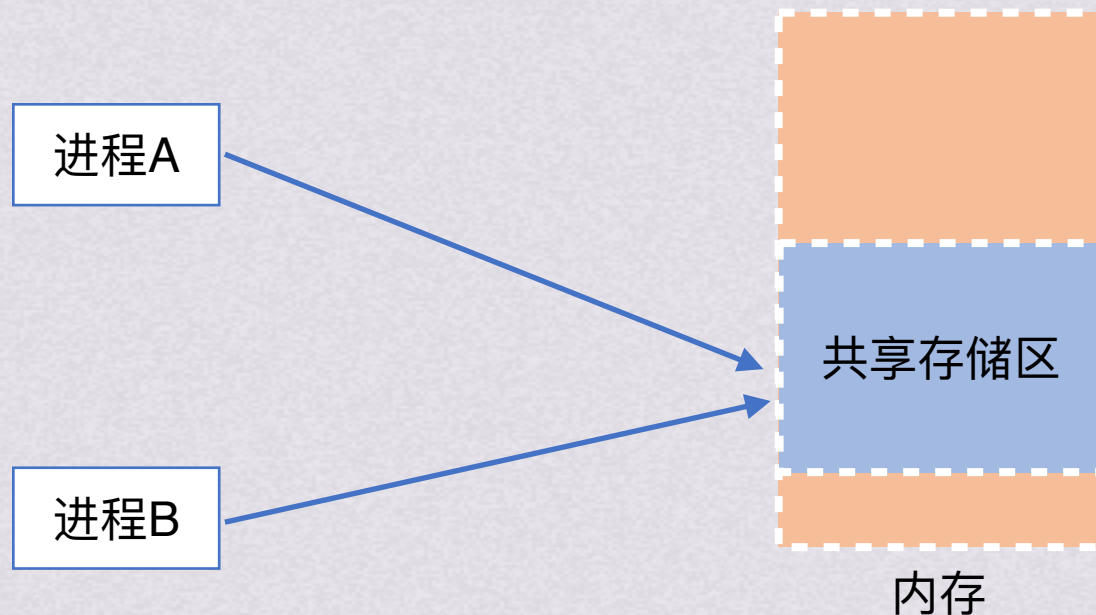
共享存储器系统中，相互通信的进程共享某些数据结构或存储区，进程之间通过这些空间进行通信

1. 基于共享数据结构的通信方式

要求各进程共享某些数据结构完成信息交换，OS仅提供共享存储器，由程序员负责对共享数据结构进行设置和对进程间同步进行处理。效率低下，属于低级进程通信

2. 基于共享存储区的通信方式

为了传送大量数据，在内存中划出一块共享存储区，各进程可通过对该区域的读写交换信息、通信，数据的形式、位置甚至访问均由进程控制，而非OS，属于高级通信





进程通信

共享存储器系统



进程通信

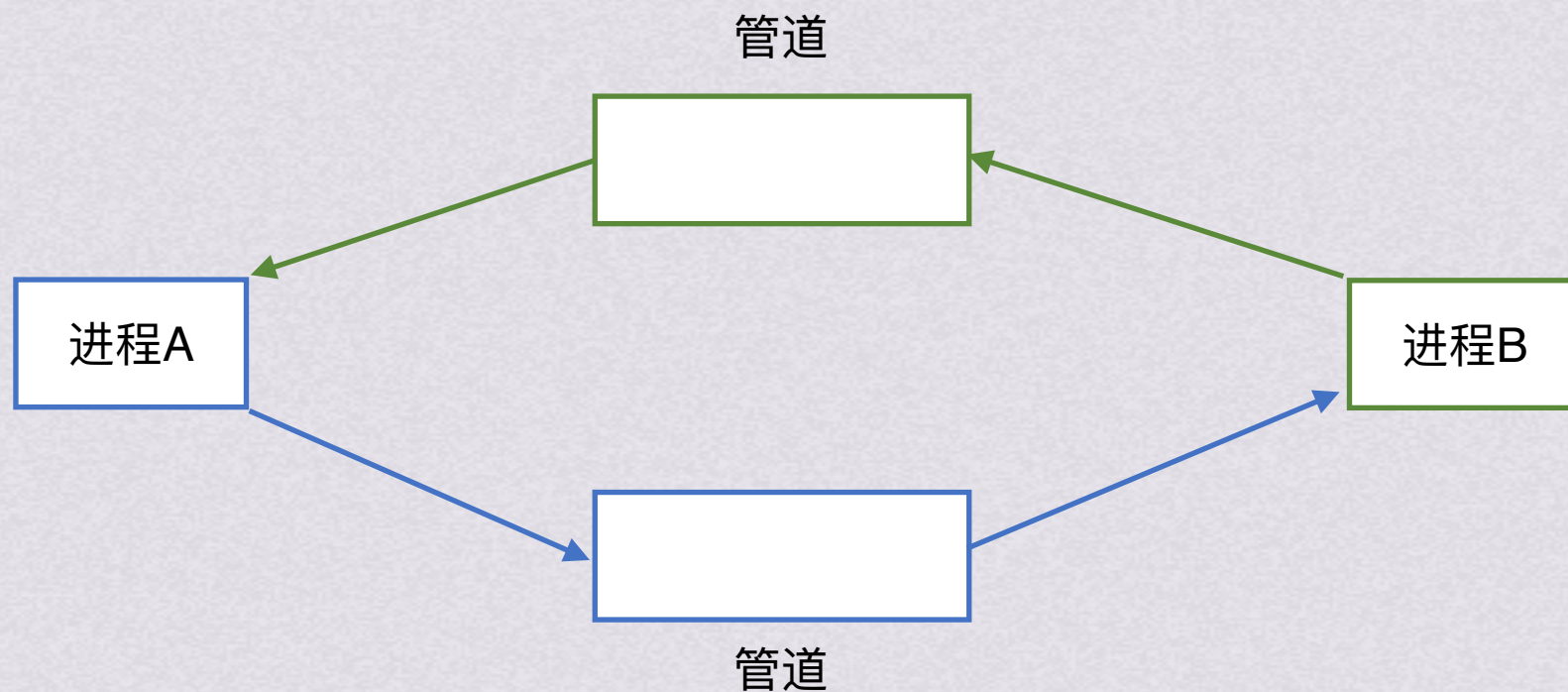
管道通信系统



进程通信

管道通信系统

“管道”指用于连接一个读进程和一个写进程以实现它们之间通信的一个共享文件，也叫pipe文件





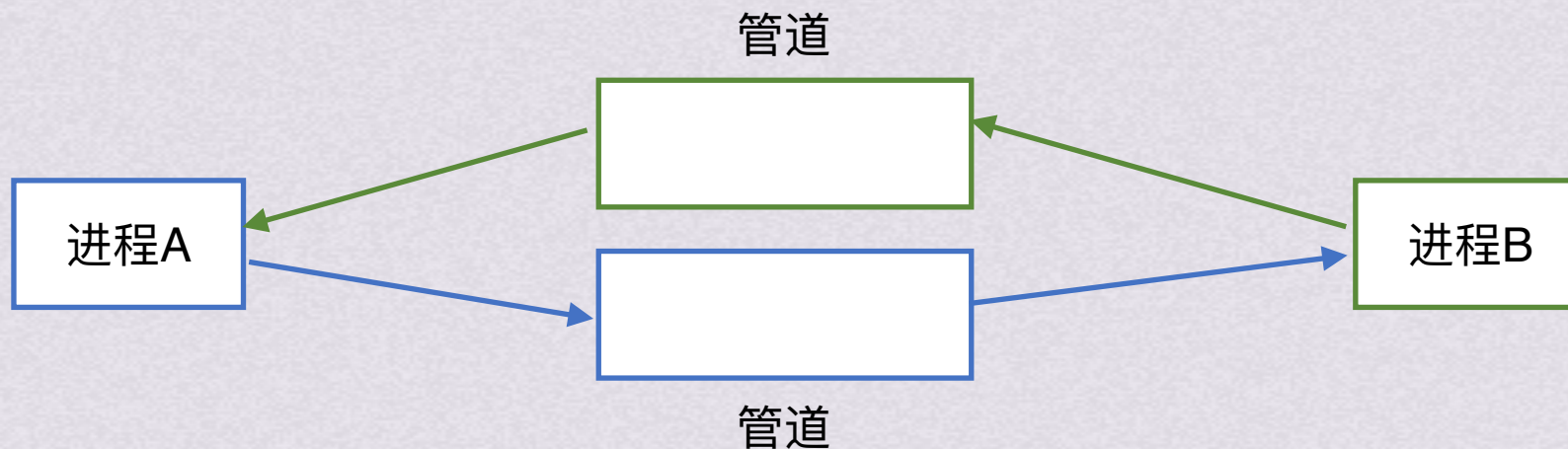
进程通信

管道通信系统

“管道”指用于连接一个读进程和一个写进程以实现它们之间通信的一个共享文件，也叫pipe文件

管道必须提供3方面的协调能力

- 1.互斥：当一个进程正在对管道执行读/写操作时，其它进程必须等待
- 2.同步：当写进程把一定数量的数据写入管道后，便等待，直到读进程取走数据后，再把它唤醒；读进程读一空管道时，也应等待
- 3.确定对方是否存在，对方已存在时才能进行通信





进程通信

管道通信系统



进程通信

消息传递系统



进程通信

消息传递系统

消息传递系统中，进程不必借助任何共享数据结构或存储区，而是会以格式化的消息为单位，将通信的数据封装在消息中

1.直接通信方式：发送进程利用OS所提供的发送原语，直接把消息发送给目标进程

2.间接通信方式：发送进程和接受进程都通过共享中间实体（信箱）进行消息的发送和接受

计网中会详细了解！



进程通信

消息传递系统